

Пълнен езан на Android

الأذان

Камбанката „аларма“ вече е истинска аларма: в часа на молитвата езанът звучи докрай през потока за аларми, телефонът вибрира с пълна сила, а на заключен екран се отваря екранът „Аларма“ със „Спри езана“ и „Заглуши звука“. Плюс: местоположение на Huawei без Google услуги, офлайн списък на градовете по света и метод „Автоматично“ според държавата.

Kotlin

AlarmManager.setAlarmClock

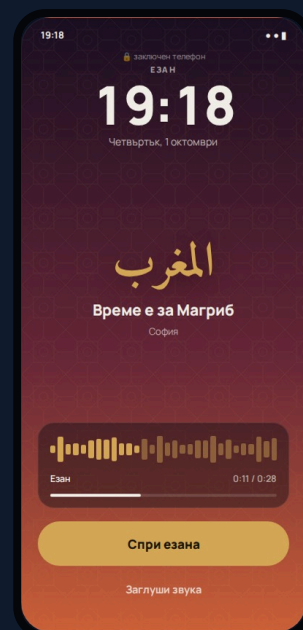
Foreground service

Екран „Аларма“

Huawei без GMS

Jest – 201 теста

22 файла



Какво е решено в този етап

Алармата е в native частта (Kotlin): setAlarmClock + foreground service + екран „Аларма“ на Kotlin (по mockup v2) · известията и напомнянията остават в expo-notifications · без „Аларми и напомняния“ – езанът е звукът на известието, както в етап 4 · вибрация 1 сек./0,7 сек. с пълна сила, докато звучи · бутоните за звука заглушават · след рестарт алармите се планират наново · „Аларма на цял екран“ и „Работа на заден план“ в Настройки · LocationManager без Google · ~25 500 града от GeoNames · 16 метода и „Автоматично“.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нов build

03 Kotlin модульт: алармата

04 Свързване с приложението

05 Настройки и „Работа на заден план“

06 Huawei без Google услуги и градовете по света

07 Метод според държавата

08 Тестове

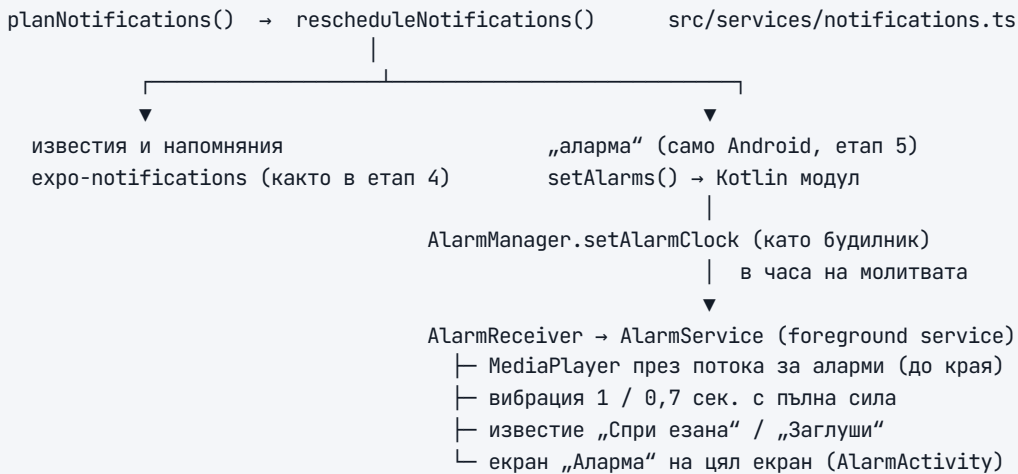
09 Проверка и край на етапа

Стъпка 1

Какво прави етапът

Планът на известията е същият като в етап 4. Новото е къде отива всяко известие: на Android известията с камбанка „аларма“ отиват в native частта и стават истински аларми.

Дърво на проекта



ЕТАП 4

ЕТАП 5

Звукът	звукът на известието, до ~30 сек.	целият езан, през потока за аларми
Спиране	махане на известието	„Спри езана“, „Заглуши“, бутоните за звука
Заклучен екран	известие	екран „Аларма“ (по mockup v2)
Вибрация	от канала – силата я решава телефонът	от кода – 1 сек./0,7 сек., пълна сила
След рестарт	expo-notifications ги възстановява	BootReceiver ги планира наново
iPhone	известие с езан (до 30 сек.)	без промяна

Ако „Аларми и напомняния“ е изключено (Android 12+), setAlarmClock не е разрешен – тогава езанът остава звукът на известието, както в етап 4. Приложението превключва само.

Стъпка 2

Нов build

Няма нови npm пакети, но native кодът е нов – нужен е нов development build. Данните на приложението остават.

Нов build на Android

```
Remove-Item -Recurse -Force android
npm expo run:android
```

- Ако Gradle даде грешка в Kotlin файл – прати ми я цялата (с името на файла и реда).
- На iPhone няма промяна в native частта.

- Шрифтовете на екрана „Аларма“ (Manrope и Amiri) са копирани в `modules/adhan-native/android/src/main/assets/adhan_fonts/` – native екранът не вижда шрифтовете на React Native.

Стъпка 3

Kotlin модулът: алармата

Всичко е в `modules/adhan-native/android/`. Екранът „Аларма“ е на Kotlin, за да се отвори веднага върху заключен екран, без да се зарежда React Native.

AndroidManifest.xml `modules/adhan-native/android/src/main`

- Разрешения: `WAKE_LOCK`, `VIBRATE`, `FOREGROUND_SERVICE(_MEDIA_PLAYBACK)`, `USE_FULL_SCREEN_INTENT`, `RECEIVE_BOOT_COMPLETED`.
- `<queries>` за `com.google.android.gms` – иначе от Android 11 приложението не вижда дали има Google услуги.
- AlarmActivity: `showWhenLocked`, `turnScreenOn`, отделна задача – не вади цялото приложение.
- BootReceiver: рестарт, обновяване, смяна на часа/часовата зона и на разрешението за точни аларми.

`modules/adhan-native/android/src/main/AndroidManifest.xml`

51 реда

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Алармата с пълния езан (етап 5) -->
  <uses-permission android:name="android.permission.WAKE_LOCK" />
  <uses-permission android:name="android.permission.VIBRATE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
  <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
  <uses-permission android:name="android.permission.USE_FULL_SCREEN_INTENT" />
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

  <!-- Android 11+: за да види дали на телефона има Google Play Services (Huawei без Google) -->
  <queries>
    <package android:name="com.google.android.gms" />
  </queries>

  <application>
    <!-- Екранът „Аларма“ върху заключен екран; отделна задача, за да не вади цялото приложение -->
    <activity
      android:name="expo.modules.adhannative.AlarmActivity"
      android:exported="false"
      android:showWhenLocked="true"
      android:turnScreenOn="true"
      android:excludeFromRecents="true"
      android:launchMode="singleInstance"
      android:taskAffinity="{applicationId}.alarm"
      android:theme="@android:style/Theme.Material.NoActionBar" />

    <!-- Пуска езана и вибрацията, докато свърши (или „Спри“) -->
    <service
      android:name="expo.modules.adhannative.AlarmService"
      android:exported="false"
      android:foregroundServiceType="mediaPlayback" />

    <!-- Будилникът на Android (setAlarmClock) вика този receiver в часа на молитвата -->
    <receiver
      android:name="expo.modules.adhannative.AlarmReceiver"
      android:exported="false" />
  </application>
</manifest>
```

```

<!-- След рестарт, обновяване на приложението, смяна на часа или на разрешението – алармите наново -->
<receiver
  android:name="expo.modules.adhannative.BootReceiver"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.BOOT_COMPLETED" />
    <action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
    <action android:name="android.intent.action.TIME_SET" />
    <action android:name="android.intent.action.TIMEZONE_CHANGED" />
    <action android:name="android.app.action.SCHEDULE_EXACT_ALARM_PERMISSION_STATE_CHANGED" />
  </intent-filter>
</receiver>
</application>
</manifest>

```

AlarmData.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Една аларма, както я праща JavaScript: час, молитва, всички текстове на езика на телефона, цветовете на фона.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmData.kt

120 реда

```

package expo.modules.adhannative

import android.graphics.Color
import org.json.JSONArray
import org.json.JSONObject

/**
 * Една аларма (езан за една молитва) – както я изпраща JavaScript частта.
 * Всички текстове идват готови от приложението (на езика на телефона),
 * за да може екранът „Аларма“ да се покаже и когато приложението е затворено.
 */
data class AlarmData(
  val id: String,
  /** Часът в милисекунди (UTC). */
  val at: Long,
  /** fajr, dhuhur, asr, maghrib, isha */
  val prayer: String,
  /** „Време е за Магриб“ */
  val title: String,
  /** „المغرب“ */
  val arabic: String,
  /** „София“ */
  val place: String,
  /** Заглавие и текст на известието. */
  val notifTitle: String,
  val notifBody: String,
  /** Фонът – трите цвята на молитвата (горе, среда, долу). */
  val colors: IntArray,
  val vibrate: Boolean,
  /** Името на звука в res/raw (без разширение): „ezan_adhan“. */
  val sound: String,
  val labels: Labels,
  /** Езикът за датата на екрана: „bg“ или „en“. */
  val lang: String,
  /** Оригиналният JSON – пази се и се праща нататък непроменен. */
  val json: String,
) {

```

```

data class Labels(
    /** „ЕЗАН“ */
    val app: String,
    /** „Спри езана“ */
    val stop: String,
    /** „Заглуши звука“ */
    val mute: String,
    /** „Заглуши“ – по-краткото за бутона в известието */
    val muteShort: String,
    /** „Затвори“ */
    val close: String,
    /** „Езан“ – името на звука под вълната */
    val soundName: String,
    /** „Аларма за молитва“ – името на канала в настройките на телефона */
    val channel: String,
)

companion object {
    fun fromJson(json: String?): AlarmData? {
        if (json.isNullOrEmpty()) return null
        return try {
            fromObject(JSONObject(json))
        } catch (e: Exception) {
            null
        }
    }

    fun fromObject(o: JSONObject): AlarmData {
        val l = o.optJSONObject("Labels") ?: JSONObject()
        val c = o.optJSONArray("colors")
        val colors = IntArray(3) { i -> parseColor(c?.optString(i), DEFAULT_COLORS[i]) }
        return AlarmData(
            id = o.getString("id"),
            at = o.getLong("at"),
            prayer = o.optString("prayer"),
            title = o.optString("title"),
            arabic = o.optString("arabic"),
            place = o.optString("place"),
            notifTitle = o.optString("notifTitle"),
            notifBody = o.optString("notifBody"),
            colors = colors,
            vibrate = o.optBoolean("vibrate", true),
            sound = o.optString("sound", "ezan_adhan"),
            Labels(
                app = l.optString("app", "ADHAN"),
                stop = l.optString("stop", "Stop"),
                mute = l.optString("mute", "Mute"),
                muteShort = l.optString("muteShort", l.optString("mute", "Mute")),
                close = l.optString("close", "Close"),
                soundName = l.optString("soundName", "Adhan"),
                channel = l.optString("channel", "Prayer alarm"),
            ),
            lang = o.optString("lang", "en"),
            json = o.toString(),
        )
    }

    fun listFromJson(json: String?): List<AlarmData> {
        if (json.isNullOrEmpty()) return emptyList()
        return try {
            val arr = JSONArray(json)

```

```

        (0 until arr.length()).mapNotNull { i ->
            try {
                fromObject(arr.getJSONObject(i))
            } catch (e: Exception) {
                null
            }
        }
    } catch (e: Exception) {
        emptyList()
    }
}

private val DEFAULT_COLORS = intArrayOf(0xFF0F1B2D.toInt(), 0xFF0F1B2D.toInt(), 0xFF122238.toInt())

private fun parseColor(s: String?, fallback: Int): Int =
    try {
        if (s.isNullOrEmpty()) fallback else Color.parseColor(s)
    } catch (e: Exception) {
        fallback
    }
}
}
}

```

AlarmScheduler.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- **setAlarmClock** – като будилника: точно, събужда телефона, минава през Doze. В лентата горе се появява иконата на будилник.
- Списъкът се пази в SharedPreferences – за рестарта. Аларма, закъсняла над 10 мин. (изключен телефон), не звъни.
- История на последните 10 аларми (планиран / истински час) – за „Проверка на известията“.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmScheduler.kt

133 реда

```

package expo.modules.adhannative

import android.app.AlarmManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.util.Log
import org.json.JSONArray
import org.json.JSONObject

/**
 * Планиране на алармите с AlarmManager.setAlarmClock – като будилника на телефона:
 * звъни точно, събужда телефона и минава през режима за пестене на батерия (Doze).
 *
 * Списъкът се пази в SharedPreferences, за да може след рестарт на телефона
 * (BootReceiver) алармите да се планират наново без приложението.
 */
object AlarmScheduler {
    private const val TAG = "AdhanAlarm"
    private const val PREFS = "adhan_alarms"
    private const val KEY_LIST = "list"
    private const val KEY_SCHEDULED = "scheduled"
    private const val KEY_HISTORY = "history"
    const val EXTRA_ALARM = "expo.modules.adhannative.ALARM"
}

```

```

const val TEST_ID = "test"

/** Аларма, по-стара от това (телефонът е бил изключен), не звъни. */
const val MAX_LATE_MS = 10 * 60 * 1000L

private fun prefs(context: Context) = context.getSharedPreferences(PREFS, Context.MODE_PRIVATE)

private fun alarmManager(context: Context) = context.getSystemService(Context.ALARM_SERVICE) as
AlarmManager

fun canScheduleExact(context: Context): Boolean =
    Build.VERSION.SDK_INT < Build.VERSION_CODES.S || alarmManager(context).canScheduleExactAlarms()

/**
 * Заменя всички аларми с новия списък (JSON масив). Отменя старите, планира бъдещите.
 * Връща колко са планирани.
 */
fun setAlarms(context: Context, json: String): Int {
    val alarms = AlarmData.listFromJson(json)
    prefs(context).edit().putString(KEY_LIST, json).apply()
    return scheduleAll(context, alarms)
}

/** След рестарт, обновяване или смяна на часа - наново от запазения списък. */
fun rescheduleAll(context: Context): Int =
    scheduleAll(context, AlarmData.listFromJson(prefs(context).getString(KEY_LIST, null)))

fun storedJson(context: Context): String = prefs(context).getString(KEY_LIST, null) ?: "[]"

private fun scheduleAll(context: Context, alarms: List<AlarmData>): Int {
    val previous = prefs(context).getStringSet(KEY_SCHEDULED, emptySet()) ?: emptySet()
    for (id in previous) cancel(context, id)

    val now = System.currentTimeMillis()
    val scheduled = mutableSetOf<String>()
    for (alarm in alarms) {
        if (alarm.at <= now) continue
        if (schedule(context, alarm)) scheduled.add(alarm.id)
    }
    prefs(context).edit().putStringSet(KEY_SCHEDULED, scheduled).apply()
    return scheduled.size
}

/** Пробна аларма след `delayMs` - не пипа списъка с истинските. */
fun scheduleTest(context: Context, json: String, delayMs: Long): Boolean {
    val alarm = AlarmData.fromJson(json) ?: return false
    val o = JSONObject(alarm.json).put("id", TEST_ID).put("at", System.currentTimeMillis() + delayMs)
    return schedule(context, AlarmData.fromObject(o))
}

private fun intentFor(context: Context, id: String): Intent =
    Intent(context, AlarmReceiver::class.java)
    // data прави всяка аларма различна за PendingIntent (extras не се броят)
    .setData(Uri.parse("adhan://alarm/$id"))

private fun schedule(context: Context, alarm: AlarmData): Boolean {
    if (!canScheduleExact(context)) return false
    val operation = PendingIntent.getBroadcast(
        context,
        0,
        intentFor(context, alarm.id).putExtra(EXTRA_ALARM, alarm.json),

```

```

        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
    )
    // при тап върху иконата на будилника в лентата горе - отваря приложението
    val launch = context.packageManager.getLaunchIntentForPackage(context.packageName) ?: Intent()
    val show = PendingIntent.getActivity(context, 0, launch, PendingIntent.FLAG_IMMUTABLE)
    return try {
        alarmManager(context).setAlarmClock(AlarmManager.AlarmClockInfo(alarm.at, show), operation)
        true
    } catch (e: SecurityException) {
        // „Аларми и напомняния“ е изключено
        Log.w(TAG, "setAlarmClock: ${e.message}")
        false
    }
}

private fun cancel(context: Context, id: String) {
    val pi = PendingIntent.getBroadcast(
        context,
        0,
        intentFor(context, id),
        PendingIntent.FLAG_NO_CREATE or PendingIntent.FLAG_IMMUTABLE,
    ) ?: return
    alarmManager(context).cancel(pi)
    pi.cancel()
}

/* ----- история (за „Проверка на известията“) */

/** Записва, че алармата е звъннала - с планирания и истинския час. Пази последните 10. */
fun recordFired(context: Context, alarm: AlarmData) {
    try {
        val arr = JSONArray(prefs(context).getString(KEY_HISTORY, null) ?: "[]")
        val item = JSONObject()
            .put("id", alarm.id)
            .put("title", alarm.notifTitle)
            .put("planned", alarm.at)
            .put("fired", System.currentTimeMillis())
        val next = JSONArray().put(item)
        for (i in 0 until minOf(arr.length(), 9)) next.put(arr.get(i))
        prefs(context).edit().putString(KEY_HISTORY, next.toString()).apply()
    } catch (e: Exception) {
        Log.w(TAG, "history: ${e.message}")
    }
}

fun historyJson(context: Context): String = prefs(context).getString(KEY_HISTORY, null) ?: "[]"
}

```

AlarmReceiver.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Пуска услугата и държи телефона буден (wake lock до 60 сек.), докато звукът тръгне.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmReceiver.kt

73 реда

```

package expo.modules.adhannative

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.os.Build

```

```

import android.os.PowerManager
import android.util.Log

/** Часът на молитвата: пуска AlarmService (звук, вибрация, известие, екран „Аларма“). */
class AlarmReceiver : BroadcastReceiver() {
    companion object {
        private var wakeLock: PowerManager.WakeLock? = null

        /**
         * Будилникът държи телефона буден само докато върви onReceive. Докато услугата
         * тръгне и пусне звука, процесорът не бива да заспи – затова собствен wake lock
         * (най-много 60 сек.); услугата го пуска, щом звукът тръгне.
         */
        @Synchronized
        fun acquireWakeLock(context: Context) {
            if (wakeLock?.isHeld == true) return
            val pm = context.getSystemService(Context.POWER_SERVICE) as PowerManager
            wakeLock = pm.newWakeLock(PowerManager.PARTIAL_WAKE_LOCK, "adhan:alarm").apply {
                setReferenceCounted(false)
                acquire(60_000L)
            }
        }

        @Synchronized
        fun releaseWakeLock() {
            try {
                wakeLock?.let { if (it.isHeld) it.release() }
            } catch (e: Exception) {
                // нищо
            }
            wakeLock = null
        }
    }

    override fun onReceive(context: Context, intent: Intent) {
        val json = intent.getStringExtra(AlarmScheduler.EXTRA_ALARM) ?: return
        val alarm = AlarmData.fromJson(json) ?: return
        // телефонът е бил изключен и алармата идва много по-късно – не звъни
        if (System.currentTimeMillis() - alarm.at > AlarmScheduler.MAX_LATE_MS) return
        acquireWakeLock(context)

        val service = Intent(context, AlarmService::class.java)
            .setAction(AlarmService.ACTION_START)
            .putExtra(AlarmScheduler.EXTRA_ALARM, json)
        try {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                context.startForegroundService(service)
            } else {
                context.startService(service)
            }
        } catch (e: Exception) {
            Log.w("AdhanAlarm", "start service: ${e.message}")
            releaseWakeLock()
        }
    }
}

/** След рестарт, обновяване на приложението или смяна на часа – алармите се планират наново. */
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        try {

```

```

        AlarmScheduler.rescheduleAll(context)
    } catch (e: Exception) {
        Log.w("AdhanAlarm", "reschedule: ${e.message}")
    }
}
}
}

```

AlarmService.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Звукът – MediaPlayer с USAGE_ALARM: звучи и при безшумен режим; спира музиката (audio focus).
- Вибрация с пълна сила (амплитуда 255) и атрибут „аларма“, докато звучи.
- Известие с „Спри езана“ и „Заглуши“ + full-screen intent към екрана „Аларма“.
- „Заглуши“ или край на езана: тихо, известието остава (вече може да се махне), екранът показва „Затвори“. Предпазно – спира след 15 мин.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmService.kt

407 реда

```

package expo.modules.adhannative

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.Context
import android.content.Intent
import android.content.pm.ServiceInfo
import android.graphics.drawable.Icon
import android.media.AudioAttributes
import android.media.AudioFocusRequest
import android.media.AudioManager
import android.media.MediaPlayer
import android.media.RingtoneManager
import android.os.Build
import android.os.Handler
import android.os.IBinder
import android.os.Looper
import android.os.PowerManager
import android.os.VibrationAttributes
import android.os.VibrationEffect
import android.os.Vibrator
import android.os.VibratorManager
import android.util.Log

/**
 * Езанът: foreground service, който пуска звука през потока за аларми
 * (звучи и при безшумен режим, както будилник), вибрира с пълна сила,
 * докато звучи, и показва известие със „Спри езана“ / „Заглуши“ и екрана „Аларма“.
 *
 * Състоянието е в companion обекта – екранът „Аларма“ (същия процес) го чете
 * няколко пъти в секунда за прогреса и вълната.
 */
class AlarmService : Service() {
    enum class State { IDLE, PLAYING, DONE }

    companion object {
        const val ACTION_START = "expo.modules.adhannative.START"
        const val ACTION_STOP = "expo.modules.adhannative.STOP"
        const val ACTION_MUTE = "expo.modules.adhannative.MUTE"
    }
}

```

```

private const val CHANNEL_ID = "adhan-alarm"
const val NOTIFICATION_ID = 7310
private const val TAG = "AdhanAlarm"
/** Предпазно: алармата спира най-много след 15 минути. */
private const val MAX_PLAY_MS = 15 * 60 * 1000L

@Volatile var current: AlarmData? = null
    private set
@Volatile var state: State = State.IDLE
    private set
@Volatile private var player: MediaPlayer? = null

/** Позиция и дължина на звука в мс (за прогреса на екрана). */
fun progress(): Pair<Int, Int> {
    val p = player ?: return Pair(0, 0)
    return try {
        Pair(p.currentPosition, p.duration)
    } catch (e: Exception) {
        Pair(0, 0)
    }
}

@Volatile var lastDuration: Int = 0
    private set

fun command(context: Context, action: String) {
    val intent = Intent(context, AlarmService::class.java).setAction(action)
    try {
        context.startService(intent)
    } catch (e: Exception) {
        // услугата вече не работи (напр. след „Заглуши“) – само чистим известието
        if (action == ACTION_STOP) {
            (context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager).cancel(NOTIFICATION_ID)
            current = null
            state = State.IDLE
        }
    }
}

private val handler = Handler(Looper.getMainLooper())
private var vibrator: Vibrator? = null
private var focusRequest: AudioFocusRequest? = null
private val timeout = Runnable { finishPlaying() }

override fun onBind(intent: Intent?): IBinder? = null

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    when (intent?.action) {
        ACTION_START -> {
            val alarm = AlarmData.fromJson(intent.getStringExtra(AlarmScheduler.EXTRA_ALARM))
            if (alarm == null) {
                // стартирана е с startForegroundService – Android иска startForeground и тук
                abortStart()
            } else {
                start(alarm)
            }
        }
        ACTION_MUTE -> finishPlaying()
        ACTION_STOP -> stopAll()
    }
}

```

```

        else -> if (state != State.PLAYING) stopSelf()
    }
    return START_NOT_STICKY
}

/* ----- старт */

private fun start(alarm: AlarmData) {
    // ако още звучи предишната аларма - спира я
    releaseSound()
    current = alarm
    state = State.PLAYING

    val notification = buildNotification(alarm, playing = true)
    try {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            startForeground(NOTIFICATION_ID, notification, ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK)
        } else {
            startForeground(NOTIFICATION_ID, notification)
        }
    } catch (e: Exception) {
        Log.w(TAG, "startForeground: ${e.message}")
    }

    if (alarm.id != AlarmScheduler.TEST_ID) AlarmScheduler.recordFired(this, alarm)
    handler.removeCallbacks(timeout)
    handler.postDelayed(timeout, MAX_PLAY_MS)
    playSound(alarm) // ако звукът не тръгне, тук срокът става 30 сек.
    if (alarm.vibrate) startVibration()
    AlarmReceiver.releaseWakeLock()
}

private fun playSound(alarm: AlarmData) {
    val attrs = AudioAttributes.Builder()
        .setUsage(AudioAttributes.USAGE_ALARM)
        .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
        .build()
    requestFocus(attrs)
    val p = MediaPlayer()
    try {
        p.setAudioAttributes(attrs)
        val resId = resources.getIdentifier(alarm.sound, "raw", packageName)
        if (resId != 0) {
            val afd = resources.openRawResourceFd(resId)
            p.setDataSource(afd.fileDescriptor, afd.startOffset, afd.length)
            afd.close()
        } else {
            // звукът липсва - сигналът за аларма на телефона
            p.setDataSource(this, RingtoneManager.getDefaultUri(RingtoneManager.TYPE_ALARM))
        }
        p.setWakeMode(this, PowerManager.PARTIAL_WAKE_LOCK)
        p.setOnCompletionListener { finishPlaying() }
        p.setOnErrorListener { _, _, _ ->
            finishPlaying()
            true
        }
        p.prepare()
        lastDuration = p.duration
        p.start()
        player = p
    } catch (e: Exception) {

```

```

        Log.w(TAG, "play: ${e.message}")
        p.release()
        // без звук – поне вибрацията и екранът; спира след 30 сек.
        handler.removeCallbacks(timeout)
        handler.postDelayed(timeout, 30_000)
    }
}

private fun requestFocus(attrs: AudioAttributes) {
    val am = getSystemService(Context.AUDIO_SERVICE) as AudioManager
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val req = AudioFocusRequest.Builder(AudioManager.AUDIOFOCUS_GAIN_TRANSIENT)
            .setAudioAttributes(attrs)
            .build()
        focusRequest = req
        am.requestAudioFocus(req)
    } else {
        @Suppress("DEPRECATION")
        am.requestAudioFocus(null, AudioManager.STREAM_ALARM, AudioManager.AUDIOFOCUS_GAIN_TRANSIENT)
    }
}

private fun abandonFocus() {
    val am = getSystemService(Context.AUDIO_SERVICE) as AudioManager
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        focusRequest?.let { am.abandonAudioFocusRequest(it) }
        focusRequest = null
    } else {
        @Suppress("DEPRECATION")
        am.abandonAudioFocus(null)
    }
}

private fun abortStart() {
    try {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val nm = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
            if (nm.getNotificationChannel(CHANNEL_ID) == null) {
                nm.createNotificationChannel(NotificationChannel(CHANNEL_ID, "Adhan",
NotificationManager.IMPORTANCE_LOW))
            }
            val n = Notification.Builder(this, CHANNEL_ID).setSmallIcon(smallIcon()).build()
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                startForeground(NOTIFICATION_ID, n, ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK)
            } else {
                startForeground(NOTIFICATION_ID, n)
            }
            stopForeground(STOP_FOREGROUND_REMOVE)
        }
    } catch (e: Exception) {
        Log.w(TAG, "abort: ${e.message}")
    }
    AlarmReceiver.releaseWakeLock()
    stopSelf()
}

/* ----- вибрация */

private fun startVibration() {
    val v: Vibrator? =
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {

```

```

        (getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibrator
    } else {
        @Suppress("DEPRECATION")
        getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
    }
    if (v == null || !v.hasVibrator()) return
    vibrator = v
    // 1 сек. вибрация / 0,7 сек. пауза, докато звучи езанът - с пълна сила (255)
    val timings = longArrayOf(0, 1000, 700)
    try {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val amplitudes = intArrayOf(0, 255, 0)
            val effect = VibrationEffect.createWaveform(timings, amplitudes, 1)
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
                v.vibrate(effect, VibrationAttributes.createForUsage(VibrationAttributes.USAGE_ALARM))
            } else {
                val attrs = AudioAttributes.Builder().setUsage(AudioAttributes.USAGE_ALARM).build()
                @Suppress("DEPRECATION")
                v.vibrate(effect, attrs)
            }
        } else {
            @Suppress("DEPRECATION")
            v.vibrate(timings, 1)
        }
    } catch (e: Exception) {
        Log.w(TAG, "vibrate: ${e.message}")
    }
}

private fun stopVibration() {
    try {
        vibrator?.cancel()
    } catch (e: Exception) {
        // нищо
    }
    vibrator = null
}

/* ----- край */

private fun releaseSound() {
    handler.removeCallbacks(timeout)
    player?.let {
        try {
            it.stop()
        } catch (e: Exception) {
            // нищо
        }
        it.release()
    }
    player = null
    stopVibration()
    abandonFocus()
}

/**
 * „Заглуши“ или езанът е свършил: тихо, без вибрация; известието остава
 * (вече може да се махне), екранът показва „Затвори“.
 */
private fun finishPlaying() {
    releaseSound()
}

```

```

    val alarm = current
    if (alarm == null) {
        stopAll()
        return
    }
    state = State.DONE
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
        stopForeground(STOP_FOREGROUND_DETACH)
    } else {
        @Suppress("DEPRECATION")
        stopForeground(false)
    }
    val nm = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
    nm.notify(NOTIFICATION_ID, buildNotification(alarm, playing = false))
    stopSelf()
}

/** „Спри езана“ / „Затвори“: всичко спира, известието и екранът изчезват. */
private fun stopAll() {
    releaseSound()
    current = null
    state = State.IDLE
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
        stopForeground(STOP_FOREGROUND_REMOVE)
    } else {
        @Suppress("DEPRECATION")
        stopForeground(true)
    }
    (getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager).cancel(NOTIFICATION_ID)
    stopSelf()
}

override fun onDestroy() {
    if (state == State.PLAYING) releaseSound()
    super.onDestroy()
}

/* ----- ИЗВЕСТИЕ ----- */

private fun ensureChannel(alarm: AlarmData) {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return
    val nm = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
    // звукът и вибрацията са от услугата, не от канала
    val channel = NotificationChannel(CHANNEL_ID, alarm.labels.channel,
NotificationManager.IMPORTANCE_HIGH)
    channel.setSound(null, null)
    channel.enableVibration(false)
    channel.lockscreenVisibility = Notification.VISIBILITY_PUBLIC
    nm.createNotificationChannel(channel)
}

private fun buildNotification(alarm: AlarmData, playing: Boolean): Notification {
    ensureChannel(alarm)
    val screen = Intent(this, AlarmActivity::class.java)
        .putExtra(AlarmScheduler.EXTRA_ALARM, alarm.json)
        .addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_NO_USER_ACTION)
    val screenPi = PendingIntent.getActivity(
        this,
        1,
        screen,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
    )
}

```

```

    )
    val stopPi = servicePending(ACTION_STOP, 2)
    val mutePi = servicePending(ACTION_MUTE, 3)

    val builder =
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            Notification.Builder(this, CHANNEL_ID)
        } else {
            @Suppress("DEPRECATION")
            Notification.Builder(this).setPriority(Notification.PRIORITY_MAX)
        }
    builder
        .setSmallIcon(smallIcon())
        .setColor(0xFFD4A857.toInt())
        .setContentTitle(alarm.notifTitle)
        .setContentText(alarm.notifBody.lineSequence().firstOrNull() ?: "")
        .setStyle(Notification.BigTextStyle().bigText(alarm.notifBody))
        .setCategory(Notification.CATEGORY_ALARM)
        .setVisibility(Notification.VISIBILITY_PUBLIC)
        .setContentIntent(screenPi)
        .setShowWhen(true)
        .setWhen(alarm.at)

    if (playing) {
        builder
            .setOngoing(true)
            .setAutoCancel(false)
            // на заключен телефон - целият екран „Аларма“ (ако е разрешено от Android 14)
            .setFullScreenIntent(screenPi, true)
            .addAction(action(alarm.labels.stop, stopPi))
            .addAction(action(alarm.labels.muteShort, mutePi))
            .setDeleteIntent(stopPi)
    } else {
        builder
            .setOngoing(false)
            .setAutoCancel(true)
            .setDeleteIntent(stopPi)
    }
    return builder.build()
}

private fun action(label: String, pi: PendingIntent): Notification.Action =
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
        Notification.Action.Builder(null as Icon?, label, pi).build()
    } else {
        @Suppress("DEPRECATION")
        Notification.Action.Builder(0, label, pi).build()
    }

private fun servicePending(action: String, code: Int): PendingIntent =
    PendingIntent.getService(
        this,
        code,
        Intent(this, AlarmService::class.java).setAction(action),
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
    )

/** Иконата на известията от app.json (expo-notifications → notification_icon). */
private fun smallIcon(): Int {
    val id = resources.getIdentifier("notification_icon", "drawable", packageName)
    return if (id != 0) id else applicationInfo.icon
}

```

```
}  
}
```

AlarmActivity.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Екранът от москир v2: градиентът на молитвата и геометричната шарка, „ЕЗАН“, часът, датата, арабското име, „Време е за ...“, мястото, вълната с прогреса, бутоните.
- Бутоните за звука заглушават; „назад“ скрива екрана, но езанът продължава (спира се от известието).

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmActivity.kt

445 реда

```
package expo.modules.adhannative  
  
import android.app.Activity  
import android.content.Context  
import android.content.Intent  
import android.content.res.ColorStateList  
import android.graphics.Canvas  
import android.graphics.Color  
import android.graphics.LinearGradient  
import android.graphics.Paint  
import android.graphics.Path  
import android.graphics.RectF  
import android.graphics.Shader  
import android.graphics.Typeface  
import android.graphics.drawable.GradientDrawable  
import android.graphics.drawable.RippleDrawable  
import android.os.Build  
import android.os.Bundle  
import android.os.Handler  
import android.os.Looper  
import android.text.TextUtils  
import android.util.TypedValue  
import android.view.Gravity  
import android.view.KeyEvent  
import android.view.View  
import android.view.ViewGroup  
import android.view.WindowManager  
import android.widget.FrameLayout  
import android.widget.LinearLayout  
import android.widget.TextView  
import java.text.SimpleDateFormat  
import java.util.Date  
import java.util.Locale  
import kotlin.math.abs  
import kotlin.math.cos  
import kotlin.math.sin  
  
/**  
 * Екранът „Аларма“ на цял екран (по одобрения москир v2): час, дата, арабското име  
 * на молитвата, „Време е за ...“, вълна с прогреса на езана, „Спри езана“ и „Заглуши звука“.  
 * Показва се и върху заключения екран. Написан е изцяло на Kotlin, за да се отвори  
 * веднага, без да се зарежда приложението.  
 */  
class AlarmActivity : Activity() {  
    private val handler = Handler(Looper.getMainLooper())  
    private var alarm: AlarmData? = null
```

```

private lateinit var clock: TextView
private lateinit var wave: WaveView
private lateinit var bar: BarView
private lateinit var timeText: TextView
private lateinit var stopBtn: TextView
private lateinit var muteBtn: TextView

private val tick = object : Runnable {
    override fun run() {
        update()
        handler.postDelayed(this, 250)
    }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O_MR1) {
        setShowWhenLocked(true)
        setTurnScreenOn(true)
    } else {
        @Suppress("DEPRECATION")
        window.addFlags(WindowManager.LayoutParams.FLAG_SHOW_WHEN_LOCKED or
WindowManager.LayoutParams.FLAG_TURN_SCREEN_ON)
    }
    window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)

    val a = AlarmService.current ?: AlarmData.fromJson(intent.getStringExtra(AlarmScheduler.EXTRA_ALARM))
    if (a == null) {
        finish()
        return
    }
    alarm = a
    setContentView(build(a))
    edgeToEdge(a)
}

override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)
    // нова аларма, докато екранът е отворен - наново
    val a = AlarmService.current ?: AlarmData.fromJson(intent.getStringExtra(AlarmScheduler.EXTRA_ALARM))
?: return
    alarm = a
    setContentView(build(a))
    edgeToEdge(a)
}

override fun onResume() {
    super.onResume()
    handler.post(tick)
}

override fun onPause() {
    super.onPause()
    handler.removeCallbacks(tick)
}

/** Бутоните за звука заглушават езана - както при будилника. */
override fun onKeyDown(keyCode: Int, event: KeyEvent?): Boolean {
    if (keyCode == KeyEvent.KEYCODE_VOLUME_UP || keyCode == KeyEvent.KEYCODE_VOLUME_DOWN) {
        if (AlarmService.state == AlarmService.State.PLAYING) {
            AlarmService.command(this, AlarmService.ACTION_MUTE)
        }
    }
}

```

```

        return true
    }
}
return super.onKeyDown(keyCode, event)
}

/* ----- обновяване */

private fun update() {
    val a = alarm ?: return
    val state = AlarmService.state
    // „Спри“ от известието или друга аларма - екранът се затваря
    if (state == AlarmService.State.IDLE || AlarmService.current?.id != a.id) {
        finish()
        return
    }
    clock.text = hm(System.currentTimeMillis())
    val playing = state == AlarmService.State.PLAYING
    val (pos, dur) = if (playing) AlarmService.progress() else Pair(AlarmService.lastDuration,
AlarmService.lastDuration)
    val fraction = if (dur > 0) pos.toFloat() / dur else 0f
    wave.playing = playing
    wave.fraction = fraction
    bar.fraction = fraction
    timeText.text = if (dur > 0) "${mmss(pos)} / ${mmss(dur)}" else ""
    if (playing) {
        stopBtn.text = a.labels.stop
        muteBtn.visibility = View.VISIBLE
    } else {
        stopBtn.text = a.labels.close
        muteBtn.visibility = View.INVISIBLE
    }
}

private fun stopAlarm() {
    AlarmService.command(this, AlarmService.ACTION_STOP)
    finish()
}

/* ----- изглед */

private fun dp(v: Float): Int = (v * resources.displayMetrics.density + 0.5f).toInt()

private fun font(name: String): Typeface =
    try {
        Typeface.createFromAsset(assets, "adhan_fonts/$name.ttf")
    } catch (e: Exception) {
        Typeface.DEFAULT_BOLD
    }

private fun text(
    value: String,
    size: Float,
    face: Typeface,
    color: Int = TEXT,
    unit: Int = TypedValue.COMPLEX_UNIT_SP,
): TextView =
    TextView(this).apply {
        text = value
        setTextSize(unit, size)
        typeface = face
    }

```

```

        setTextColor(color)
        gravity = Gravity.CENTER
        includeFontPadding = false
    }

    private fun build(a: AlarmData): View {
        val extraBold = font("manrope_extrabold")
        val bold = font("manrope_bold")
        val semi = font("manrope_semibold")
        val regular = font("manrope_regular")
        val amiri = font("amiri_bold")

        val root = FrameLayout(this)
        root.addView(BackgroundView(this, a.colors), FrameLayout.LayoutParams(MATCH, MATCH))

        val column = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            gravity = Gravity.CENTER_HORIZONTAL
        }
        root.addView(column, FrameLayout.LayoutParams(MATCH, MATCH))

        // ---- горе: ЕЗАН, часът, датата
        val top = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            gravity = Gravity.CENTER_HORIZONTAL
            setPadding(0, dp(26f), 0, 0)
        }
        top.addView(text(a.labels.app.uppercase(), 11f, extraBold, alpha(TEXT, 0.65f)).apply { letterSpacing = 0.16f })
        clock = text(hm(System.currentTimeMillis()), 72f, extraBold, TEXT, TypedValue.COMPLEX_UNIT_DIP).apply {
            letterSpacing = -0.03f
            setPadding(0, dp(2f), 0, dp(2f))
        }
        top.addView(clock)
        top.addView(text(dateLine(a), 14f, regular, alpha(TEXT, 0.75f)))
        column.addView(top, LinearLayout.LayoutParams(MATCH, WRAP))

        // ---- среда: арабското име, „Време е за ...“, мястото
        val mid = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            gravity = Gravity.CENTER
            setPadding(dp(20f), 0, dp(20f), 0)
        }
        mid.addView(
            text(a.arabic, 64f, amiri, GOLD, TypedValue.COMPLEX_UNIT_DIP).apply {
                setShadowLayer(dp(15f).toFloat(), 0f, dp(4f).toFloat(), 0x59000000)
                includeFontPadding = true
            },
        )
        mid.addView(
            text(a.title, 21f, extraBold).apply {
                setPadding(0, dp(6f), 0, 0)
                maxLines = 2
                ellipsize = TextUtils.TruncateAt.END
            },
        )
        if (a.place.isNotEmpty()) {
            mid.addView(text(a.place, 13f, regular, alpha(TEXT, 0.7f)).apply { setPadding(0, dp(6f), 0, 0) })
        }
        column.addView(mid, LinearLayout.LayoutParams(MATCH, 0, 1f))
    }

```

```

// ---- картата със звука: Вълна, име и време, прогрес
val card = LinearLayout(this).apply {
    orientation = LinearLayout.VERTICAL
    setPadding(dp(16f), dp(16f), dp(16f), dp(14f))
    background = GradientDrawable().apply {
        cornerRadius = dp(22f).toFloat()
        setColor(Color.rgb(102, 6, 10, 20))
        setStroke(dp(1f), Color.rgb(20, 255, 255, 255))
    }
}
wave = WaveView(this)
card.addView(wave, LinearLayout.LayoutParams(MATCH, dp(38f)))
val trackRow = LinearLayout(this).apply {
    orientation = LinearLayout.HORIZONTAL
    setPadding(0, dp(12f), 0, dp(12f))
}
trackRow.addView(
    text(a.labels.soundName, 12.5f, semi).apply { gravity = Gravity.START },
    LinearLayout.LayoutParams(0, WRAP, 1f),
)
timeText = text("", 12.5f, semi, alpha(TEXT, 0.65f)).apply { gravity = Gravity.END }
trackRow.addView(timeText, LinearLayout.LayoutParams(WRAP, WRAP))
card.addView(trackRow, LinearLayout.LayoutParams(MATCH, WRAP))
bar = BarView(this)
card.addView(bar, LinearLayout.LayoutParams(MATCH, dp(4f)))
column.addView(
    card,
    LinearLayout.LayoutParams(MATCH, WRAP).apply {
        leftMargin = dp(22f)
        rightMargin = dp(22f)
    },
)

// ---- бутоните
val actions = LinearLayout(this).apply {
    orientation = LinearLayout.VERTICAL
    setPadding(dp(22f), dp(18f), dp(22f), dp(30f))
}
stopBtn = text(a.labels.stop, 17f, extraBold, GOLD_INK).apply {
    background = pill(GOLD, 0x33000000)
    isClickable = true
    setOnClickListener { stopAlarm() }
}
actions.addView(stopBtn, LinearLayout.LayoutParams(MATCH, dp(60f)))
muteBtn = text(a.labels.mute, 14.5f, bold, alpha(TEXT, 0.85f)).apply {
    background = pill(Color.TRANSPARENT, 0x33FFFFFF)
    isClickable = true
    setOnClickListener { AlarmService.command(this@AlarmActivity, AlarmService.ACTION_MUTE) }
}
actions.addView(muteBtn, LinearLayout.LayoutParams(MATCH, dp(46f)).apply { topMargin = dp(8f) })
column.addView(actions, LinearLayout.LayoutParams(MATCH, WRAP))

// отстъп за лентите на системата (часовник горе, навигация долу)
column.setOnApplyWindowInsetsListener { v, insets ->
    @Suppress("DEPRECATION")
    v.setPadding(0, insets.systemWindowInsetTop, 0, insets.systemWindowInsetBottom)
    insets
}
return root
}

```

```

private fun pill(color: Int, ripple: Int): RippleDrawable {
    val shape = GradientDrawable().apply {
        cornerRadius = dp(999f).toFloat()
        setColor(color)
    }
    val mask = GradientDrawable().apply {
        cornerRadius = dp(999f).toFloat()
        setColor(Color.WHITE)
    }
    return RippleDrawable(ColorStateList.valueOf(ripple), shape, mask)
}

/** Фонът минава под лентите на системата; те са прозрачни. */
private fun edgeToEdge(a: AlarmData) {
    @Suppress("DEPRECATION")
    window.decorView.systemUiVisibility =
        View.SYSTEM_UI_FLAG_LAYOUT_STABLE or View.SYSTEM_UI_FLAG_LAYOUT_FULLSCREEN or
View.SYSTEM_UI_FLAG_LAYOUT_HIDE_NAVIGATION
    @Suppress("DEPRECATION")
    window.statusBarColor = Color.TRANSPARENT
    @Suppress("DEPRECATION")
    window.navigationBarColor = Color.TRANSPARENT
    window.decorView.setBackgroundColor(a.colors[2])
}

/* ----- формати */

private fun hm(ms: Long): String = SimpleDateFormat("HH:mm", Locale.US).format(Date(ms))

private fun mmss(ms: Int): String {
    val s = ms / 1000
    return "${s / 60}:${(s % 60).toString().padStart(2, '0')}"
}

/** „Четвъртък, 1 октомври“ / „Thursday, 1 October“ */
private fun dateLine(a: AlarmData): String {
    val locale = if (a.lang == "bg") Locale("bg") else Locale.UK
    val s = SimpleDateFormat("EEEE, d MMMM", locale).format(Date(a.at))
    return s.replaceFirstChar { it.titlecase(locale) }
}

companion object {
    private const val MATCH = ViewGroup.LayoutParams.MATCH_PARENT
    private const val WRAP = ViewGroup.LayoutParams.WRAP_CONTENT
    val TEXT = Color.parseColor("#F2EFE8")
    val GOLD = Color.parseColor("#D4A857")
    val GOLD_INK = Color.parseColor("#1A1408")

    fun alpha(color: Int, a: Float): Int = Color.argb((a * 255).toInt(), Color.red(color),
Color.green(color), Color.blue(color))
}

/** Градиентът на молитвата + геометричната шарка (8-лъчева звезда) със 7,5% злато. */
private class BackgroundView(context: Context, private val colors: IntArray) : View(context) {
    private val fill = Paint()
    private val line = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        style = Paint.Style.STROKE
        color = AlarmActivity.alpha(AlarmActivity.GOLD, 0.075f)
        strokeWidth = context.resources.displayMetrics.density
    }
}

```

```

private val tile = Path()

override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
    fill.shader = LinearGradient(0f, 0f, 0f, h.toFloat(), colors, floatArrayOf(0f, 0.55f, 1f),
    Shader.TileMode.CLAMP)
    val u = context.resources.displayMetrics.density // 1 единица = 1 dp, плочката е 60 dp
    tile.reset()
    fun seg(x1: Float, y1: Float, x2: Float, y2: Float) {
        tile.moveTo(x1 * u, y1 * u)
        tile.lineTo(x2 * u, y2 * u)
    }
    tile.addRect(16 * u, 16 * u, 44 * u, 44 * u, Path.Direction.CW)
    tile.moveTo(30 * u, 10.2f * u)
    tile.lineTo(49.8f * u, 30 * u)
    tile.lineTo(30 * u, 49.8f * u)
    tile.lineTo(10.2f * u, 30 * u)
    tile.close()
    seg(0f, 0f, 16f, 16f)
    seg(60f, 0f, 44f, 16f)
    seg(0f, 60f, 16f, 44f)
    seg(60f, 60f, 44f, 44f)
    seg(30f, 0f, 30f, 10.2f)
    seg(30f, 49.8f, 30f, 60f)
    seg(0f, 30f, 10.2f, 30f)
    seg(49.8f, 30f, 60f, 30f)
    tile.addCircle(30 * u, 30 * u, 6 * u, Path.Direction.CW)
}

override fun onDraw(canvas: Canvas) {
    canvas.drawRect(0f, 0f, width.toFloat(), height.toFloat(), fill)
    val step = 60 * context.resources.displayMetrics.density
    var y = 0f
    while (y < height) {
        var x = 0f
        while (x < width) {
            canvas.save()
            canvas.translate(x, y)
            canvas.drawPath(tile, line)
            canvas.restore()
            x += step
        }
        y += step
    }
}

/** Вълната: 30 златни черти, които „дишат“, докато звучи; изминалата част е по-ярка. */
private class WaveView(context: Context) : View(context) {
    var playing = true
    var fraction = 0f
    private val paint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = AlarmActivity.GOLD }
    private val rect = RectF()
    private val heights = FloatArray(BARS) { w -> (0.35f + 0.65f * abs(sin(w * 1.7) * cos(w *
    0.45))).toFloat() }

    override fun onDraw(canvas: Canvas) {
        val gap = 3 * context.resources.displayMetrics.density
        val barW = (width - gap * (BARS - 1)) / BARS
        val t = System.currentTimeMillis() / 1300.0 * 2 * Math.PI
        for (i in 0 until BARS) {
            val scale =

```

```

        if (playing) {
            val phase = (sin(t + i * 0.55) + 1) / 2 // 0..1
            (0.18f + (heights[i] - 0.18f) * phase.toFloat())
        } else {
            0.12f
        }
        val h = height * scale
        val x = i * (barW + gap)
        rect.set(x, (height - h) / 2, x + barW, (height + h) / 2)
        val played = (i + 0.5f) / BARS <= fraction
        paint.alpha = if (!playing) 128 else if (played) 255 else 115
        canvas.drawRoundRect(rect, barW / 2, barW / 2, paint)
    }
    if (playing) postInvalidateOnAnimation()
}

companion object {
    private const val BARS = 30
}

}

/** Лентата на прогреса под вълната. */
private class BarView(context: Context) : View(context) {
    var fraction = 0f
    set(v) {
        field = v.coerceIn(0f, 1f)
        invalidate()
    }
    private val track = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Color.argb(36, 255, 255, 255) }
    private val fill = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = AlarmActivity.TEXT }
    private val rect = RectF()

    override fun onDraw(canvas: Canvas) {
        val r = height / 2f
        rect.set(0f, 0f, width.toFloat(), height.toFloat())
        canvas.drawRoundRect(rect, r, r, track)
        rect.set(0f, 0f, width * fraction, height.toFloat())
        canvas.drawRoundRect(rect, r, r, fill)
    }
}
}

```

AdhanNativeModule.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Функциите за JavaScript: setAlarms, getAlarms, getAlarmHistory, testAlarm, canUseFullScreenIntent, isIgnoringBatteryOptimizations, hasGoogleServices, getCurrentLocation и отварянето на настройките.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AdhanNativeModule.kt

149 реда

```

package expo.modules.adhannative

import android.app.AlarmManager
import android.app.NotificationManager
import android.content.ActivityNotFoundException
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.PowerManager
import android.provider.Settings
import expo.modules.kotlin.Promise

```

```

import expo.modules.kotlin.exception.Exceptions
import expo.modules.kotlin.modules.Module
import expo.modules.kotlin.modules.ModuleDefinition

/**
 * Native модулът на Езан (само Android):
 * - етап 4: точни известия (Android 12+);
 * - етап 5: алармата с пълния езан (AlarmManager.setAlarmClock + foreground service +
 * екран „Аларма“), аларма на цял екран (Android 14+), работа на заден план
 * (оптимизация на батерията) и местоположение без Google услуги (Huawei).
 */
class AdhanNativeModule : Module() {
    private val context: Context
    get() = appContext.reactContext ?: throw Exceptions.ReactContextLost()

    private val packageUri: Uri
    get() = Uri.parse("package:${context.packageName}")

    private fun open(intent: Intent, fallback: Intent? = null) {
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        try {
            context.startActivity(intent)
        } catch (e: ActivityNotFoundException) {
            val f = fallback ?: Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
            context.startActivity(f.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK))
        }
    }

    override fun definition() = ModuleDefinition {
        Name("AdhanNative")

        /* ----- точни аларми (етап 4) */

        // Може ли приложението да планира точни аларми. До Android 12 – винаги да.
        // От Android 14 разрешението „Аларми и напомняния“ е изключено по подразбиране.
        Function("canScheduleExactAlarms") {
            if (Build.VERSION.SDK_INT < Build.VERSION_CODES.S) {
                true
            } else {
                val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
                alarmManager.canScheduleExactAlarms()
            }
        }

        // Отваря системния екран „Аларми и напомняния“ за Езан.
        // Ако производителят го е махнал – екрана с информация за приложението.
        Function("openExactAlarmSettings") {
            val intent =
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
                    Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM, packageUri)
                } else {
                    Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri)
                }
            open(intent)
        }

        /* ----- алармата с езана (етап 5) */

        // Заменя всички аларми (JSON масив, виж AlarmData). Връща колко са планирани.
        Function("setAlarms") { json: String ->
            AlarmScheduler.setAlarms(context, json)
        }
    }
}

```

```

}

// Запазеният списък (JSON) - за „Проверка на известията“.
Function("getAlarms") {
    AlarmScheduler.storedJson(context)
}

// Последните звъннали аларми (JSON): планиран и истински час.
Function("getAlarmHistory") {
    AlarmScheduler.historyJson(context)
}

// Пробна аларма след delayMs.
Function("testAlarm") { json: String, delayMs: Double ->
    AlarmScheduler.scheduleTest(context, json, delayMs.toLong())
}

Function("stopAlarm") {
    AlarmService.command(context, AlarmService.ACTION_STOP)
}

/* ----- аларма на цял екран (Android 14+) */

Function("canUseFullScreenIntent") {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        true
    } else {
        val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        nm.canUseFullScreenIntent()
    }
}

Function("openFullScreenIntentSettings") {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        open(Intent(Settings.ACTION_MANAGE_APP_USE_FULL_SCREEN_INTENT, packageUri))
    } else {
        open(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri))
    }
}

/* ----- работа на заден план */

// true - телефонът не ограничава Езан (оптимизацията на батерията е изключена за него).
Function("isIgnoringBatteryOptimizations") {
    val pm = context.getSystemService(Context.POWER_SERVICE) as PowerManager
    pm.isIgnoringBatteryOptimizations(context.packageName)
}

// Настройките на Езан в телефона (оттам: Батерия → Без ограничения).
Function("openAppSettings") {
    open(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS, packageUri))
}

// Списъкът „Оптимизация на батерията“ на Android.
Function("openBatteryOptimizationSettings") {
    open(Intent(Settings.ACTION_IGNORE_BATTERY_OPTIMIZATIONS_SETTINGS))
}

/* ----- местоположение без Google (Huawei) */

Function("hasGoogleServices") {

```

```

    NativeLocation.hasGoogleServices(context)
  }

  AsyncFunction("getCurrentLocation") { timeoutMs: Double, promise: Promise ->
    val ctx = context
    if (!NativeLocation.hasPermission(ctx)) {
      promise.resolve(null)
      return@AsyncFunction
    }
    NativeLocation.current(ctx, timeoutMs.toLong()) { location ->
      promise.resolve(location?.let { NativeLocation.toMap(it) })
    }
  }
}
}
}

```

index.ts modules/adhan-native

- Обвивката за JavaScript – на iPhone, в уеб и със стар build връща безопасни стойности.

modules/adhan-native/index.ts

166 реда

```

import { requireOptionalNativeModule } from 'expo';
import { Platform } from 'react-native';

/**
 * Native функциите на Езан (само Android). На iPhone и в уеб модулът липсва –
 * тогава функциите връщат безопасни стойности (null / false).
 */
interface AdhanNativeModule {
  canScheduleExactAlarms(): boolean;
  openExactAlarmSettings(): void;
  // етап 5
  setAlarms?(json: string): number;
  getAlarms?(): string;
  getAlarmHistory?(): string;
  testAlarm?(json: string, delayMs: number): boolean;
  stopAlarm?(): void;
  canUseFullScreenIntent?(): boolean;
  openFullScreenIntentSettings?(): void;
  isIgnoringBatteryOptimizations?(): boolean;
  openAppSettings?(): void;
  openBatteryOptimizationSettings?(): void;
  hasGoogleServices?(): boolean;
  getCurrentLocation?(timeoutMs: number): Promise<NativeLocation | null>;
}

export interface NativeLocation {
  latitude: number;
  longitude: number;
  accuracy: number | null;
  timestamp: number;
  provider: string | null;
}

/** Една аларма за native частта (виж AlarmData.kt). */
export interface NativeAlarm {
  id: string;
  /** ms */
  at: number;
}

```

```

prayer: string;
/** „Време е за Магриб“ */
title: string;
arabic: string;
place: string;
notifTitle: string;
notifBody: string;
/** Градиентът на молитвата - 3 цвята „#RRGGBB“. */
colors: [string, string, string];
vibrate: boolean;
/** Звукът в res/raw без разширение. */
sound: string;
lang: 'bg' | 'en';
labels: {
  app: string;
  stop: string;
  mute: string;
  muteShort: string;
  close: string;
  soundName: string;
  channel: string;
};
}

export interface AlarmHistoryItem {
  id: string;
  title: string;
  planned: number;
  fired: number;
}

const native = Platform.OS === 'android' ? requireOptionalNativeModule<AdhanNativeModule>('AdhanNative') :
null;

function safe<T>(fn: () => T, fallback: T): T {
  try {
    return fn();
  } catch {
    return fallback;
  }
}

/** null - неприложимо (iPhone, уеб) или модулът липсва (стар build без него). */
export function canScheduleExactAlarms(): boolean | null {
  if (!native) return null;
  return safe(() => native.canScheduleExactAlarms(), null);
}

export function openExactAlarmSettings(): void {
  safe(() => native?.openExactAlarmSettings(), undefined);
}

/* ----- алармата с езана */

/** Има ли native аларма (Android с build от етап 5). Иначе езанът е звукът на известието. */
export function hasNativeAlarms(): boolean {
  return !!native?.setAlarms;
}

/** Заменя всички аларми. Връща колко са планирани (-1 - няма native аларма). */
export function setAlarms(alarms: NativeAlarm[]): number {

```

```

    if (!native?.setAlarms) return -1;
    return safe(() => native.setAlarms!(JSON.stringify(alarms)), -1);
}

export function getAlarms(): NativeAlarm[] {
    if (!native?.getAlarms) return [];
    return safe(() => JSON.parse(native.getAlarms!()) as NativeAlarm[], []);
}

export function getAlarmHistory(): AlarmHistoryItem[] {
    if (!native?.getAlarmHistory) return [];
    return safe(() => JSON.parse(native.getAlarmHistory!()) as AlarmHistoryItem[], []);
}

export function testAlarm(alarm: NativeAlarm, delayMs: number): boolean {
    if (!native?.testAlarm) return false;
    return safe(() => native.testAlarm!(JSON.stringify(alarm), delayMs), false);
}

export function stopAlarm(): void {
    safe(() => native?.stopAlarm?.(), undefined);
}

/* ----- аларма на цял екран */

/** Android 14+: разрешено ли е „Аларма на цял екран“. null - неприложимо. */
export function canUseFullScreenIntent(): boolean | null {
    if (!native?.canUseFullScreenIntent) return null;
    return safe(() => native.canUseFullScreenIntent!(), null);
}

export function openFullScreenIntentSettings(): void {
    safe(() => native?.openFullScreenIntentSettings?.(), undefined);
}

/* ----- работа на заден план */

/** true - Езан работи без ограничения на батерията. null - неприложимо. */
export function isIgnoringBatteryOptimizations(): boolean | null {
    if (!native?.isIgnoringBatteryOptimizations) return null;
    return safe(() => native.isIgnoringBatteryOptimizations!(), null);
}

export function openAppSettings(): void {
    safe(() => native?.openAppSettings?.(), undefined);
}

export function openBatteryOptimizationSettings(): void {
    safe(() => native?.openBatteryOptimizationSettings?.(), undefined);
}

/* ----- местоположение без Google */

/** false - телефон без Google услуги (Huawei); null - неприложимо/неизвестно. */
export function hasGoogleServices(): boolean | null {
    if (!native?.hasGoogleServices) return null;
    return safe(() => native.hasGoogleServices!(), null);
}

/** Позицията от вграждения в Android LocationManager (без Google). null - няма. */
export async function getNativeLocation(timeoutMs: number): Promise<NativeLocation | null> {

```

```

    if (!native?.getCurrentLocation) return null;
    try {
      return await native.getCurrentLocation(timeoutMs);
    } catch {
      return null;
    }
  }
}

```

Стъпка 4

Свързване с приложението

Планът е същият; новото е разделянето.

notifications.ts src/services

- `nativeAlarmsEnabled()` – Android, build с етап 5 и точните аларми не са забранени.
- Камбанка „аларма“ → `toNativeAlarm()` → `setAlarms()`; останалите – expo-notifications.
- Състоянието има `fullScreen`, `battery` и `alarms`.
- Пробният бутон „Аларма“ пуска истинската аларма след 10 сек.

src/services/notifications.ts

425 реда

```

import * as Notifications from 'expo-notifications';
import { Linking, Platform } from 'react-native';
import { create } from 'zustand';

import {
  canScheduleExactAlarms,
  canUseFullScreenIntent,
  hasNativeAlarms,
  isIgnoringBatteryOptimizations,
  openExactAlarmSettings,
  setAlarms,
  testAlarm,
  type NativeAlarm,
} from '../modules/adhan-native';

import { formatHM } from '@domain/format';
import { planNotifications, type PlannedNotification, type SoundKind } from '@domain/notifications';
import { PRAYERS, type PrayerId } from '@domain/prayers';
import { getI18n } from '@/i18n';
import { PHASE_GRADIENTS } from '@theme/gradients';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectLocation, selectTimesOptions, useSettings } from '@store/settings';

/**
 * Известията за намаз – само локални (без интернет, без Firebase/FCM и без HMS Push).
 * Android планира всяко известие с AlarmManager, iPhone – с UNUserNotificationCenter.
 * Затова работят еднакво на телефони с Google услуги, на Huawei без тях и на iPhone.
 */

export type NotificationPermission = 'granted' | 'denied' | 'undetermined' | 'unavailable';

interface NotificationStatus {
  /** false, докато разрешението още не е проверено (първите мигове след стартиране). */
  checked: boolean;
  permission: NotificationPermission;
}

```

```

/** Android 13+/iOS: може ли приложението пак да покаже системния въпрос. */
canAskAgain: boolean;
/** Android 12+: точни известия („Аларми и напомняния“); null – неприложимо. */
exact: boolean | null;
/** Колко известия са планирани и до кога (заедно с алармите). */
count: number;
until: Date | null;
/** Android 14+: „Аларма на цял екран“; null – неприложимо. */
fullScreen: boolean | null;
/** Android: true – без ограничения на батерията; null – неприложимо. */
battery: boolean | null;
/** Колко аларми с езан са планирани в native частта (Android). */
alarms: number;
}

export const useNotificationStatus = create<NotificationStatus>(() => ({
  checked: false,
  permission: Platform.OS === 'web' ? 'unavailable' : 'undetermined',
  canAskAgain: true,
  exact: null,
  count: 0,
  until: null,
  fullScreen: null,
  battery: null,
  alarms: 0,
}));

const SUPPORTED = Platform.OS === 'android' || Platform.OS === 'ios';

/** Файловете от assets/sounds (виж app.json → expo-notifications → sounds). */
const SOUND_FILE: Record<SoundKind, string> = {
  chime: 'ezan_chime.wav',
  adhan: 'ezan_adhan.wav',
};

/* ----- канали (Android) */

type ChannelKind = 'prayer' | 'adhan' | 'reminder';

/**
 * На Android звукът и вибрацията са на канала и не могат да се сменят след създаването му.
 * Затова каналите с и без вибрация са различни („adhan-v2“ / „adhan-s2“),
 * а ненужният се трие при смяна на настройката.
 * Версия 2: каналът „Езан“ звучи като аларма (виж setupChannels).
 * Версия 3: при „Езан“ вибрацията продължава, докато звучи езанът; при известията – една.
 * Каналите от по-старите версии се трият.
 */
const CHANNEL_VERSION = 3;
function channelId(kind: ChannelKind, vibrate: boolean, version = CHANNEL_VERSION): string {
  return `${kind}-${vibrate ? 'v' : 's'}${version}`;
}

/** Известие и напомняне: едно дълго вибриране (1 сек.). */
const VIBRATION_ONCE = [0, 1000];

/**
 * Езан: 1 сек. вибрация / 0,7 сек. пауза, докато звучи временният езан (~28 сек.).
 * Силата на вибрацията се определя от телефона (Настройки → Звуци и вибрация).
 * В етап 5 алармата управлява вибрацията сама – със силата и продължителността на пълния езан.
 */
const VIBRATION_ADHAN = [0, ...Array.from({ length: 16 }, () => [1000, 700])).flat()];

```

```

const vibrationFor = (kind: ChannelKind) => (kind === 'adhan' ? VIBRATION_ADHAN : VIBRATION_ONCE);

function channelFor(n: PlannedNotification): ChannelKind {
  if (n.kind === 'reminder' || n.kind === 'refresh') return 'reminder';
  return n.sound === 'adhan' ? 'adhan' : 'prayer';
}

let channelsFor: boolean | null = null;

async function setupChannels(vibrate: boolean): Promise<void> {
  if (Platform.OS !== 'android' || channelsFor === vibrate) return;
  const { t } = getI18n();
  const { AndroidAudioUsage: Usage, AndroidAudioContentType: Content, AndroidImportance: Imp } =
Notifications;
  const defs: {
    kind: ChannelKind;
    name: string;
    sound: SoundKind;
    importance: Notifications.AndroidImportance;
    usage: Notifications.AndroidAudioUsage;
  }[] = [
    { kind: 'prayer', name: t.notifications.channelPrayer, sound: 'chime', importance: Imp.HIGH, usage:
Usage.NOTIFICATION },
    // Езанът звучи през потока за аларми: не зависи от силата на звука за известия
    // и от безшумния режим – както будилник. (Пълният езан с „Спри“ – етап 5.)
    { kind: 'adhan', name: t.notifications.channelAdhan, sound: 'adhan', importance: Imp.MAX, usage:
Usage.ALARM },
    { kind: 'reminder', name: t.notifications.channelReminder, sound: 'chime', importance: Imp.HIGH, usage:
Usage.NOTIFICATION },
  ];
  for (const d of defs) {
    await Notifications.setNotificationChannelAsync(channelId(d.kind, vibrate), {
      name: d.name,
      importance: d.importance,
      sound: SOUND_FILE[d.sound],
      audioAttributes: { usage: d.usage, contentType: Content.SONIFICATION },
      enableVibrate: vibrate,
      vibrationPattern: vibrate ? vibrationFor(d.kind) : null,
      lockscreenVisibility: Notifications.AndroidNotificationVisibility.PUBLIC,
    });
    await Notifications.deleteNotificationChannelAsync(channelId(d.kind, !vibrate));
    // каналите от по-стари версии
    for (let v = 1; v < CHANNEL_VERSION; v++) {
      await Notifications.deleteNotificationChannelAsync(channelId(d.kind, true, v));
      await Notifications.deleteNotificationChannelAsync(channelId(d.kind, false, v));
    }
  }
  channelsFor = vibrate;
}

/* ----- разрешение */

function toPermission(p: Notifications.NotificationPermissionsStatus): NotificationPermission {
  if (Platform.OS === 'ios') {
    const s = p.ios?.status;
    if (
      s === Notifications.IosAuthorizationStatus.AUTHORIZED ||
      s === Notifications.IosAuthorizationStatus.PROVISIONAL ||
      s === Notifications.IosAuthorizationStatus.EPHEMERAL
    ) {

```

```

        return 'granted';
    }
    return s === Notifications.IosAuthorizationStatus.DENIED ? 'denied' : 'undetermined';
}
if (p.granted) return 'granted';
return p.status === 'undetermined' ? 'undetermined' : 'denied';
}

export async function refreshPermission(): Promise<NotificationPermission> {
    if (!SUPPORTED) return 'unavailable';
    const p = await Notifications.getPermissionsAsync();
    const permission = toPermission(p);
    useNotificationStatus.setState({
        checked: true,
        permission,
        canAskAgain: p.canAskAgain,
        exact: canScheduleExactAlarms(),
        fullScreen: canUseFullScreenIntent(),
        battery: isIgnoringBatteryOptimizations(),
    });
    return permission;
}

/**
 * Пита за разрешение. Ако системата вече не показва въпроса (отказано завинаги) -
 * отваря настройките на приложението в телефона.
 */
export async function requestPermission(): Promise<NotificationPermission> {
    if (!SUPPORTED) return 'unavailable';
    const current = await Notifications.getPermissionsAsync();
    if (toPermission(current) === 'denied' && !current.canAskAgain) {
        await Linking.openSettings();
        return 'denied';
    }
    const p = await Notifications.requestPermissionsAsync({
        ios: { allowAlert: true, allowSound: true, allowBadge: false },
    });
    const permission = toPermission(p);
    useNotificationStatus.setState({ permission, canAskAgain: p.canAskAgain });
    if (permission === 'granted') rescheduleNotifications();
    return permission;
}

export { openExactAlarmSettings };

/* ----- планиране */

/** Всичко, което определя едно известие. Ако се смени, известието се пренасрочва. */
function signatureOf(n: PlannedNotification, channel: string, exact: boolean | null): string {
    return `${n.at.getTime()}|${n.sound}|${channel}|${exact}|${n.title}|${n.body}`;
}

let running: Promise<void> | null = null;
let again = false;

/**
 * Привежда планираните известия в съответствие с настройките.
 * Вика се при стартиране, при връщане в приложението, при всяка промяна на мястото,
 * часовете, камбанките, напомнянето и вибрацията, и от фоновата задача.
 * Паралелните извиквания се обединяват.
 */

```

```

export function rescheduleNotifications(): Promise<void> {
  if (running) {
    again = true;
    return running;
  }
  running = doReschedule()
    .catch((e) => console.warn('[notifications]', e))
    .finally(() => {
      running = null;
      if (again) {
        again = false;
        rescheduleNotifications();
      }
    });
  return running;
}

async function doReschedule(): Promise<void> {
  if (!SUPPORTED) return;
  const permission = await refreshPermission();
  const exact = useNotificationStatus.getState().exact;

  const s = useSettings.getState();
  const { lang, t } = getI18n();
  const plan =
    permission === 'granted'
      ? planNotifications({
          now: new Date(),
          options: selectTimesOptions(s).options,
          modes: useAlertPrefs.getState().modes,
          reminderMinutes: s.reminderMinutes,
          placeName: selectLocation(s).names[lang],
          texts: { prayers: t.prayers, ...t.notifications },
        })
      : [];

  await setupChannels(s.vibrate);

  // Android (етап 5): езанът е истинска аларма в native частта - пълен звук, „Спри“,
  // екран „Аларма“. Без точни аларми (setAlarmClock не става) - остава звукът на известието.
  let alarmCount = 0;
  let notifications = plan;
  if (nativeAlarmsEnabled(exact)) {
    const alarms = plan.filter(isAlarm);
    notifications = plan.filter((n) => !isAlarm(n));
    alarmCount = Math.max(0, setAlarms(alarms.map((n) => toNativeAlarm(n, s.vibrate,
selectLocation(s).names[lang]))));
  } else if (hasNativeAlarms()) {
    setAlarms([]);
  }

  // Сравнява с вече планираните: трие само променените и липсващите, добавя само новите.
  const existing = await Notifications.getAllScheduledNotificationsAsync();
  const ours = new Map(
    existing
      .filter((r) => r.identifier.startsWith('ezan-'))
      .map((r) => [r.identifier, (r.content.data as { sig?: string } | null)?.sig ?? '']),
  );

  const wanted = notifications.map((n) => {
    const channel = channelId(channelFor(n), s.vibrate);

```

```

    return { n, channel, sig: signatureOf(n, channel, exact) };
});
const wantedIds = new Set(wanted.map((w) => w.n.id));

for (const [id, sig] of ours) {
    const w = wanted.find((x) => x.n.id === id);
    if (!wantedIds.has(id) || !w || w.sig !== sig) {
        await Notifications.cancelScheduledNotificationAsync(id);
        ours.delete(id);
    }
}

for (const { n, channel, sig } of wanted) {
    if (ours.has(n.id)) continue;
    await Notifications.scheduleNotificationAsync({
        identifier: n.id,
        content: {
            title: n.title,
            body: n.body,
            // на Android звукът е от канала, но без име на звук известието би било „тихо“
            sound: SOUND_FILE[n.sound],
            // за Android под 8; на по-новите вибрацията е от канала
            vibrate: s.vibrate ? vibrationFor(channelFor(n)) : undefined,
            data: { sig, kind: n.kind, prayer: n.prayer, sound: n.sound, at: n.at.getTime() },
            ...(Platform.OS === 'android' ? { color: '#D4A857' } : {}),
        },
        trigger: {
            type: Notifications.SchedulableTriggerInputTypes.DATE,
            date: n.at,
            channelId: channel,
        },
    });
}

useNotificationStatus.setState({
    count: plan.length,
    until: plan.length ? plan[plan.length - 1].at : null,
    alarms: alarmCount,
});
}

/* ----- алармата (Android, етап 5) */

const isAlarm = (n: PlannedNotification) => n.kind === 'prayer' && n.sound === 'adhan' && n.prayer !== null;

/** Native алармата се ползва, ако я има в build-a и точните аларми не са забранени. */
function nativeAlarmsEnabled(exact: boolean | null): boolean {
    return Platform.OS === 'android' && hasNativeAlarms() && exact !== false;
}

/** Едно известие от плана → аларма за native частта, с всички текстове на езика на телефона. */
function toNativeAlarm(n: PlannedNotification, vibrate: boolean, place: string): NativeAlarm {
    const { lang, t } = getI18n();
    const prayer = (n.prayer ?? 'dhuhr') as PrayerId;
    const name = t.prayers[prayer];
    return {
        id: n.id,
        at: n.at.getTime(),
        prayer,
        title: t.alarm.title(name),
    };
}

```

```

    arabic: PRAYERS[prayer].arabic,
    place,
    notifTitle: n.title,
    notifBody: n.body,
    colors: [...PHASE_GRADIENTS[prayer]] as [string, string, string],
    vibrate,
    sound: SOUND_FILE.adhan.replace(/\.wav$/, ''),
    lang,
    labels: {
      app: t.alarm.app,
      stop: t.alarm.stop,
      mute: t.alarm.mute,
      muteShort: t.alarm.muteShort,
      close: t.alarm.close,
      soundName: t.alarm.soundName,
      channel: t.alarm.channel,
    },
  };
}

/**
 * Пробна аларма след 10 сек. в native частта (екранът „Аларма“ и пълният звук).
 * false - няма native аларма (тогава - пробното известие със звука на езана).
 */
function sendTestAlarm(at: number): boolean {
  if (!nativeAlarmsEnabled(useNotificationStatus.getState().exact)) return false;
  const { t } = getI18n();
  const s = useSettings.getState();
  const place = selectLocation(s).names[getI18n().lang];
  const n: PlannedNotification = {
    id: 'test',
    at: new Date(at),
    kind: 'prayer',
    prayer: 'maghrib',
    sound: 'adhan',
    title: `${t.alarm.testTitle} - ${formatHM(new Date(at))}`,
    body: t.notifications.testBody,
  };
  return testAlarm(toNativeAlarm(n, s.vibrate, place), at - Date.now());
}

/** Трие всички известия на Езан и ги планира наново (от „Проверка на известията“). */
export async function resetAllNotifications(): Promise<void> {
  if (!SUPPORTED) return;
  await running;
  const existing = await Notifications.getAllScheduledNotificationsAsync();
  for (const r of existing) {
    if (r.identifier.startsWith('ezan-')) await
Notifications.cancelScheduledNotificationAsync(r.identifier);
  }
  channelsFor = null;
  await rescheduleNotifications();
}

/** Пробно известие след 10 сек. - с кратък сигнал или с езан. Връща часа, в който ще дойде. */
export async function sendTestNotification(sound: SoundKind): Promise<number> {
  const at = Date.now() + 10_000;
  if (!SUPPORTED) return at;
  if (sound === 'adhan' && sendTestAlarm(at)) return at;
  const s = useSettings.getState();
  await setupChannels(s.vibrate);
}

```

```

const { t } = getI18n();
await Notifications.scheduleNotificationAsync({
  identifier: `test-${Date.now()}`,
  content: {
    title: t.notifications.testTitle,
    body: t.notifications.testBody,
    sound: SOUND_FILE[sound],
    vibrate: s.vibrate ? vibrationFor(sound === 'adhan' ? 'adhan' : 'prayer') : undefined,
    data: { kind: 'test', sound, at },
  },
  trigger: {
    type: Notifications.SchedulableTriggerInputTypes.DATE,
    date: new Date(at),
    channelId: channelId(sound === 'adhan' ? 'adhan' : 'prayer', s.vibrate),
  },
});
return at;
}

/** Как се показва известие, докато приложението е отворено: като обикновено, със звук. */
export function configureNotificationHandler(): void {
  if (!SUPPORTED) return;
  Notifications.setNotificationHandler({
    handleNotification: async () => ({
      shouldShowBanner: true,
      shouldShowList: true,
      shouldPlaySound: true,
      shouldSetBadge: false,
    }),
  });
}

```

diagnostics.tsx src/app

- Нови отметки: „Аларма на цял екран“ и „Работа на заден план“ (с „Поправи“).
- Алармите се виждат в „Следващи“ (злато), а звънелите – в „Последни“ с точността.

src/app/diagnostics.tsx

518 реда

```

import * as Notifications from 'expo-notifications';
import { router } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { ActivityIndicator, Platform, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { getAlarmHistory, getAlarms, openFullScreenIntentSettings } from '../../modules/adhan-native';

import { GeometricPattern } from '@components/GeometricPattern';
import { AlertIcon, ChevronDownIcon, ChevronIcon } from '@components/icons';
import { formatGregorianShort, formatHM } from '@domain/format';
import type { SoundKind } from '@domain/notifications';
import { useI18n } from '@i18n';
import {
  openExactAlarmSettings,
  refreshPermission,
  requestPermission,
  resetAllNotifications,
  sendTestNotification,
  useNotificationStatus,
} from '@services/notifications';

```

```

import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

/* ----- данни */

type Kind = 'alarm' | 'notification' | 'reminder' | 'other';

interface Snapshot {
  /** Кога са прочетени данните - показва се под заглавието. */
  loadedAt: Date;
  channels: Notifications.NotificationChannel[];
  upcoming: { id: string; at: Date; title: string; kind: Kind }[];
  recent: { id: string; planned: Date | null; shown: Date; title: string }[];
}

const pad = (n: number) => String(n).padStart(2, '0');
const hms = (d: Date) => `${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`;
const USAGE: Record<number, string> = { 4: 'alarm', 5: 'notification' };

const atOf = (data: unknown) => {
  const at = (data as { at?: number } | null)?.at;
  return typeof at === 'number' ? new Date(at) : null;
};

/** Видът на известието по канала му (Android) или по данните (iPhone). */
function kindOf(r: Notifications.NotificationRequest): Kind {
  const channel = (r.trigger as { channelId?: string } | null)?.channelId ?? '';
  const data = r.content.data as { kind?: string; sound?: string } | null;
  if (channel.startsWith('adhan') || data?.sound === 'adhan') return 'alarm';
  if (channel.startsWith('reminder') || data?.kind === 'reminder' || data?.kind === 'refresh') return 'reminder';
  if (channel.startsWith('prayer') || data?.kind === 'prayer') return 'notification';
  return 'other';
}

async function fetchSnapshot(): Promise<Snapshot> {
  if (Platform.OS === 'web') return { loadedAt: new Date(), channels: [], upcoming: [], recent: [] };
  await refreshPermission();
  const [channels, scheduled, presented] = await Promise.all([
    Platform.OS === 'android' ? Notifications.getNotificationChannelsAsync() : Promise.resolve([]),
    Notifications.getAllScheduledNotificationsAsync(),
    Notifications.getPresentedNotificationsAsync(),
  ]);
  // Android, етап 5: алармите с езана са в native частта, не в expo-notifications
  const now = Date.now();
  const alarms = getAlarms()
    .filter((a) => a.at > now)
    .map((a) => ({ id: a.id, at: new Date(a.at), title: a.notifTitle, kind: 'alarm' as Kind }));
  const fired = getAlarmHistory().map((h) => ({
    id: `alarm-${h.id}-${h.fired}`,
    planned: new Date(h.planned),
    shown: new Date(h.fired),
    title: h.title,
  }));
  return {
    loadedAt: new Date(),
    channels: channels.filter((c) => !c.id.startsWith('expo_')),
    upcoming:
      .map((r) => ({ id: r.identifier, at: atOf(r.content.data) ?? new Date(0), title: r.content.title ?? '', kind: kindOf(r) }))
      .concat(alarms)
  };
}

```

```

        .sort((a, b) => a.at.getTime() - b.at.getTime()),
recent: presented
    .map((n) => ({
        id: n.request.identifier,
        planned: atOf(n.request.content.data),
        shown: new Date(n.date),
        title: n.request.content.title ?? '',
    }))
    .concat(fired)
    .sort((a, b) => b.shown.getTime() - a.shown.getTime()),
};
}

/* ----- екран */

/**
 * „Проверка на известията“ (Настройки → Известия):
 * състояние с отметки, пробно известие, последните показани (със закъснението),
 * следващите планирани и – сгънати – техническите данни.
 */
export default function DiagnosticsScreen() {
    const insets = useSafeAreaInsets();
    const { t } = useI18n();
    const d = t.notifications.diag;
    const status = useNotificationStatus();
    const [snap, setSnap] = useState<Snapshot | null>(null);
    const [busy, setBusy] = useState(false);
    const [techOpen, setTechOpen] = useState(false);
    const [test, setTest] = useState<{ sound: SoundKind; at: number } | null>(null);
    const [now, setNow] = useState(() => Date.now());

    const [refreshing, setRefreshing] = useState(false);
    const [resetDone, setResetDone] = useState(false);
    const load = useCallback(() => fetchSnapshot().then(setSnap), []);

    // „Обнови“: чете наново от телефона; индикаторът стои поне 0,6 сек., за да се забележи
    const refresh = async () => {
        setRefreshing(true);
        await Promise.all([load(), new Promise((r) => setTimeout(r, 600))]);
        setRefreshing(false);
    };

    useEffect(() => {
        let alive = true;
        fetchSnapshot().then((x) => alive && setSnap(x));
        return () => {
            alive = false;
        };
    }, []);

    // обратно броене на пробното известие; след него – обновяване, за да се види в „Последни“
    useEffect(() => {
        if (!test) return;
        const timer = setInterval(() => {
            const t2 = Date.now();
            setNow(t2);
            if (t2 >= test.at + 1500) {
                setTest(null);
                load();
            }
        }, 500);
    });

```

```

    return () => clearInterval(timer);
}, [test, load]);

const closing = useRef(false);
const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
};

const sendTest = async (sound: SoundKind) => {
    const at = await sendTestNotification(sound);
    setNow(at - 10_000);
    setTest({ sound, at });
};

const reset = async () => {
    setBusy(true);
    setResetDone(false);
    await resetAllNotifications();
    await load();
    setBusy(false);
    setResetDone(true);
};

const granted = status.permission === 'granted';
const kindLabel: Record<Kind, string> = {
    alarm: d.kindAlarm,
    notification: d.kindNotification,
    reminder: d.kindReminder,
    other: d.kindOther,
};
const until = status.until ? `${formatGregorianShort(status.until, t.date)}, ${formatHM(status.until)}` :
'';
const device =
    Platform.OS === 'android'
        ? `${Platform.constants.Manufacturer} ${Platform.constants.Model} · Android
        ${Platform.constants.Release} (API ${Platform.Version})`
        : `${Platform.OS} ${Platform.Version}`;

const testButton = (sound: SoundKind, label: string) => {
    const active = test?.sound === sound;
    const left = active ? Math.max(0, Math.ceil((test.at - now) / 1000)) : 0;
    return (
        <Pressable
            onPress={() => sendTest(sound)}
            disabled={!test || !granted}
            accessibilityRole="button"
            style={({ pressed }) => [
                styles.testBtn,
                sound === 'adhan' && styles.testBtnGold,
                (!test && !active) || !granted ? styles.disabled : null,
                pressed && styles.pressed,
            ]}
        >
            <AlertIcon mode={sound === 'adhan' ? 'adhan' : 'notify'} size={20} color={sound === 'adhan' ?
colors.goldInk : colors.text} />
            <Text style={[styles.testLabel, sound === 'adhan' && styles.testLabelGold]}>{label}</Text>
            {active && <Text style={[styles.testIn, sound === 'adhan' && styles.testLabelGold]}>
{d.testIn(left)}</Text>}
        </Pressable>
    );
};

```

```

    </Pressable>
  );
};

return (
  <View style={styles.root}>
    <GeometricPattern />
    <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
      <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
        <ChevronIcon direction="left" size={18} color={colors.gold} />
        <Text style={styles.link}>{t.settings.title}</Text>
      </Pressable>
      <Pressable onPress={refresh} disabled={refreshing} hitSlop={10} accessibilityRole="button" style=
{styles.refreshBtn}>
        {refreshing ? <ActivityIndicator size="small" color={colors.gold} /> : <Text style={styles.link}>
{d.refresh}</Text>}
      </Pressable>
    </View>

    <ScrollView contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}>
      <View style={styles.titleBox}>
        <Text style={styles.title}>{d.title}</Text>
        <Text style={styles.subtitle}>{d.subtitle}</Text>
        {snap && <Text style={styles.updated}>{d.updated(hms(snap.loadedAt))}</Text>}
      </View>

      {/* ---- Состояние ---- */}
      <Section label={d.status}>
        <Check
          ok={granted}
          text={granted ? d.permOk : d.permBad}
          action={granted ? undefined : { label: d.allow, onPress: requestPermission }}
          first
        />
        {status.exact !== null && (
          <Check
            ok={status.exact}
            text={status.exact ? d.exactOk : d.exactBad}
            action={status.exact ? undefined : { label: d.allow, onPress: openExactAlarmSettings }}
          />
        )}
        {granted && status.fullScreen !== null && (
          <Check
            ok={status.fullScreen}
            text={status.fullScreen ? d.fullScreenOk : d.fullScreenBad}
            action={status.fullScreen ? undefined : { label: d.allow, onPress:
openFullScreenIntentSettings }}
          />
        )}
        {granted && status.battery !== null && (
          <Check
            ok={status.battery}
            text={status.battery ? d.batteryOk : d.batteryBad}
            action={status.battery ? undefined : { label: d.fix, onPress: () => router.push('/battery')}
          }
        )}

        <Check ok={status.count > 0} text={status.count > 0 ? d.scheduledOk(status.count, until) :
d.scheduledNone} />
      </Section>
    </ScrollView>
  </View>
);

```

```

    { /* ---- Пробно известие ---- */
    <Section label={d.test}>
      <View style={styles.testBox}>
        <View style={styles.testRow}>
          {testButton('chime', d.testNotification)}
          {testButton('adhan', d.testAlarm)}
        </View>
        <Text style={styles.small}>{d.testHint}</Text>
      </View>
    </Section>

    {!snap ? (
      <ActivityIndicator color={colors.gold} style={{ marginTop: 12 }} />
    ) : (
      <>
        { /* ---- Последни ---- */
        <Section label={d.recent}>
          {snap.recent.length === 0 ? (
            <Line first>
              <Text style={styles.small}>{d.recentNone}</Text>
            </Line>
          ) : (
            snap.recent.slice(0, 8).map((p, i) => {
              const delay = p.planned ? Math.round((p.shown.getTime() - p.planned.getTime()) / 1000) :
null;

              const onTime = delay !== null && delay < 60;
              return (
                <Line key={p.id + i} first={i === 0}>
                  <View style={styles.item}>
                    <Text style={styles.itemTitle} numberOfLines={1}>
                      {p.title}
                    </Text>
                    <Text style={styles.small}>
                      {p.planned ? d.plannedShown(hms(p.planned), hms(p.shown)) : hms(p.shown)}
                    </Text>
                  </View>
                  {delay !== null && (
                    <View style={[styles.badge, onTime ? styles.badgeOk : styles.badgeWarn]}>
                      <Text style={[styles.badgeText, onTime ? styles.badgeTextOk :
styles.badgeTextWarn]}>
                        {onTime ? d.onTime : d.late(delay)}
                      </Text>
                    </View>
                  )}
                </Line>
              );
            })
          )}
        </Section>

        { /* ---- Следващи ---- */
        {snap.upcoming.length > 0 && (
          <Section label={d.upcoming}>
            {snap.upcoming.slice(0, 8).map((r, i) => (
              <Line key={r.id} first={i === 0}>
                <View style={styles.timeCol}>
                  <Text style={styles.time}>{formatHM(r.at)}</Text>
                  <Text style={styles.tiny}>{t.date.weekdaysShort[r.at.getDay()]}</Text>
                </View>
                <Text style={[styles.itemTitle, styles.flex]} numberOfLines={1}>
                  {r.title}
                </Text>
              </Line>
            ))
          </Section>
        )}
        </Section>
      </>
    )}
  )}

```

```

        </Text>
        <View style={[styles.tag, r.kind === 'alarm' && styles.tagGold]}>
            <Text style={[styles.tagText, r.kind === 'alarm' && styles.tagTextGold]}>
{kindLabel[r.kind]}</Text>
        </View>
    </Line>
    )}}
</Section>
})

{/* ---- Технически данни (сгънати) ---- */}
<View style={styles.card}>
    <Pressable
        onPress={() => setTechOpen((o) => !o)}
        accessibilityRole="button"
        accessibilityState={{ expanded: techOpen }}
        style={styles.techHead}
    >
        <Text style={styles.itemTitle}>{d.tech}</Text>
        <View style={techOpen && styles.flip}>
            <ChevronDownIcon color={colors.muted} />
        </View>
    </Pressable>
    {techOpen && (
        <View style={styles.techBody}>
            <Text style={styles.small}>
                {d.device}: {device}
            </Text>
            {snap.channels.map((c) => (
                <Text key={c.id} style={styles.small}>
                    <Text style={styles.mono}>{c.id}</Text> -{' '}
                    {d.channelLine(
                        c.importance,
                        c.sound ?? '-',
                        c.enableVibrate,
                        USAGE[c.audioAttributes?.usage ?? -1] ?? String(c.audioAttributes?.usage ?? '-'),
                    )}
                </Text>
            ))}
        </View>
    )}
</View>

    <View style={styles.resetBox}>
        <Pressable onPress={reset} disabled={busy} accessibilityRole="button" style=
{styles.resetBtn}>
            {busy ? <ActivityIndicator color={colors.gold} /> : <Text style={styles.link}>{d.reset}
</Text>}
        </Pressable>
        <Text style={[styles.small, styles.center]}>{resetDone ? d.resetDone : d.resetHint}</Text>
    </View>
</>
    )}
</ScrollView>
</View>
);
}

/* ----- части */

function Section({ label, children }: { label: string; children: React.ReactNode }) {

```

```

return (
  <View style={styles.section}>
    <Text style={styles.sectionLabel}>{label.toUpperCase()}</Text>
    <View style={styles.card}>{children}</View>
  </View>
);
}

function Line({ children, first }: { children: React.ReactNode; first?: boolean }) {
  return <View style={[styles.line, !first && styles.lineBorder]}>{children}</View>;
}

function Check({
  ok,
  text,
  action,
  first,
}: {
  ok: boolean;
  text: string;
  action?: { label: string; onPress: () => void };
  first?: boolean;
}) {
  return (
    <Line first={first}>
      <View style={[styles.dot, ok ? styles.dotOk : styles.dotWarn]}>
        <Text style={styles.dotText}>{ok ? '✓' : '!'}</Text>
      </View>
      <Text style={styles.checkText, !ok && styles.checkWarn}>{text}</Text>
      {action && (
        <Pressable onPress={action.onPress} hitSlop={8} accessibilityRole="button" style=
{styles.actionBtn}>
          <Text style={styles.actionText}>{action.label}</Text>
        </Pressable>
      )}
    </Line>
  );
}

const GREEN = '#5FBF8F';

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: {
    paddingHorizontal: 16,
    paddingBottom: 4,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6 },
  link: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 20 },
  titleBox: { gap: 6, marginTop: 4 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
  subtitle: { fontFamily: fonts.regular, fontSize: 13, lineHeight: 19, color: colors.muted },

  section: { gap: 7 },
  sectionLabel: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted,
paddingLeft: 6 },
  card: {

```

```

    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.06)',
    borderRadius: 18,
    overflow: 'hidden',
  },
  line: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12,
minHeight: 52 },
  lineBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },

  dot: { width: 24, height: 24, borderRadius: 12, alignItems: 'center', justifyContent: 'center' },
  dotOk: { backgroundColor: 'rgba(95,191,143,0.18)' },
  dotWarn: { backgroundColor: colors.warn },
  dotText: { fontFamily: fonts.extrabold, fontSize: 13, color: colors.text, includeFontPadding: false },
  checkText: { flex: 1, fontFamily: fonts.semibold, fontSize: 14, lineHeight: 19, color: colors.text },
  checkWarn: { color: colors.warn },
  actionBtn: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 999, backgroundColor: colors.gold
},
  actionText: { fontFamily: fonts.bold, fontSize: 12.5, color: colors.goldInk },

  testBox: { padding: 14, gap: 12 },
  testRow: { flexDirection: 'row', gap: 10 },
  testBtn: {
    flex: 1,
    height: 78,
    borderRadius: 16,
    alignItems: 'center',
    justifyContent: 'center',
    gap: 4,
    backgroundColor: 'rgba(255,255,255,0.08)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.10)',
  },
  testBtnGold: { backgroundColor: colors.gold, borderColor: colors.gold },
  testLabel: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
  testLabelGold: { color: colors.goldInk },
  testIn: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.textDim, ...tabularNums },
  disabled: { opacity: 0.4 },
  pressed: { transform: [{ scale: 0.97 }] },

  item: { flex: 1, gap: 2 },
  flex: { flex: 1 },
  itemTitle: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
  small: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted },
  tiny: { fontFamily: fonts.medium, fontSize: 10.5, color: colors.muted },
  mono: { fontFamily: fonts.bold, color: colors.textDim },

  badge: { paddingHorizontal: 9, paddingVertical: 4, borderRadius: 999 },
  badgeOk: { backgroundColor: 'rgba(95,191,143,0.16)' },
  badgeWarn: { backgroundColor: 'rgba(227,154,75,0.18)' },
  badgeText: { fontFamily: fonts.bold, fontSize: 11.5, ...tabularNums },
  badgeTextOk: { color: GREEN },
  badgeTextWarn: { color: colors.warn },

  timeCol: { width: 46, alignItems: 'flex-start' },
  time: { fontFamily: fonts.extrabold, fontSize: 15, color: colors.text, ...tabularNums },
  tag: { paddingHorizontal: 8, paddingVertical: 3, borderRadius: 999, backgroundColor:
'rgba(255,255,255,0.08)' },
  tagGold: { backgroundColor: 'rgba(212,168,87,0.18)' },
  tagText: { fontFamily: fonts.semibold, fontSize: 11, color: colors.textDim },
  tagTextGold: { color: colors.gold },

```

```

techHead: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  paddingHorizontal: 14,
  paddingVertical: 14,
},
flip: { transform: [{ rotate: '180deg' }] },
techBody: {
  paddingHorizontal: 14,
  paddingBottom: 14,
  gap: 6,
  borderTopWidth: StyleSheet.hairlineWidth,
  borderTopColor: 'rgba(255,255,255,0.10)',
  paddingTop: 10,
},
resetBox: { gap: 6, alignItems: 'center' },
resetBtn: {
  alignSelf: 'stretch',
  alignItems: 'center',
  paddingVertical: 13,
  borderRadius: 14,
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.45)',
},
center: { textAlign: 'center', paddingHorizontal: 12 },
refreshBtn: { minWidth: 70, alignItems: 'flex-end' },
updated: { fontFamily: fonts.medium, fontSize: 11.5, color: colors.muted, ...tabularNums },
});

```

Стъпка 5

Настройки и „Работа на заден план“

По одобрения mockup.

NotificationsGroup.tsx src/components/settings

- „Точни известия“, „Аларма на цял екран“ (Android 14+) – „Разрешени/Изключени“ с обяснение и връзка, когато са изключени.
- „Работа на заден план“ → екрана с инструкциите.

src/components/settings/NotificationsGroup.tsx

172 реда

```

import { router } from 'expo-router';
import { useState } from 'react';
import { Platform, Pressable, StyleSheet, Text, View } from 'react-native';

import { formatGregorianShort, formatHM } from '@/domain/format';
import { useI18n } from '@/i18n';
import { openFullScreenIntentSettings } from '../../modules/adhan-native';

import { openExactAlarmSettings, requestPermission, useNotificationStatus } from
'@/services/notifications';
import { REMINDER_CHOICES, useSettings } from '@/store/settings';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

```

```

import { Choice, Group, NavRow, Row, SwitchRow, settingsStyles } from './controls';

/** Грyпaтa „Извeстия“ в „Нaстрoйки“: пoзвoлeниe, тoчнa извeстия, нaпoмнянe, вибpaция, пpoбa. */
export function NotificationsGroup() {
  const { t } = useI18n();
  const n = t.notifications;
  const status = useNotificationStatus();
  const reminderMinutes = useSettings((s) => s.reminderMinutes);
  const setReminderMinutes = useSettings((s) => s.setReminderMinutes);
  const vibrate = useSettings((s) => s.vibrate);
  const setVibrate = useSettings((s) => s.setVibrate);
  const [open, setOpen] = useState(false);

  if (status.permission === 'unavailable') {
    return (
      <Group label={n.group}>
        <Row first>
          <Text style={styles.text}>{n.webNote}</Text>
        </Row>
      </Group>
    );
  }

  const granted = status.permission === 'granted';
  const permissionText = granted ? n.granted : status.permission === 'denied' ? n.denied : n.undetermined;

  const scheduledNote = !granted
    ? n.deniedNote
    : status.count === 0
    ? n.scheduledNone
    : n.scheduled(
        status.count,
        status.until ? `${formatGregorianShort(status.until, t.date)}, ${formatHM(status.until)}` : '',
      );

  const reminderLabel = (m: number) => (m === 0 ? n.reminderOff : n.reminderValue(m));

  return (
    <Group label={n.group} note={scheduledNote}>
      <Row first>
        <Text style={settingsStyles.label}>{n.status}</Text>
        <Text style={[styles.value, !granted && styles.warn]}>{permissionText}</Text>
      </Row>
      {!granted && (
        <Pressable onPress={requestPermission} accessibilityRole="button" style={styles.linkRow}>
          <Text style={styles.link}>{n.allow}</Text>
        </Pressable>
      )}

      {granted && status.exact !== null && (
        <StatusRow
          label={n.exact}
          ok={status.exact}
          okText={n.exactOn}
          badText={n.exactOff}
          note={n.exactNote}
          action={{ label: n.exactAllow, onPress: openExactAlarmSettings }}
        />
      )}
    </Group>
  );

  /** Android, этап 5: алармата на цял екран и работата на заден план */
}

```

```

{granted && status.fullScreen != null && (
  <StatusRow
    label={n.fullScreen}
    ok={status.fullScreen}
    okText={n.fullScreenOn}
    badText={n.fullScreenOff}
    note={n.fullScreenNote}
    action={{ label: n.fullScreenAllow, onPress: openFullScreenIntentSettings }}
  />
)}
{granted && status.battery != null && (
  <NavRow
    label={n.background}
    value={status.battery ? n.backgroundOk : n.backgroundBad}
    warn={!status.battery}
    onPress={() => router.push('/battery')}
  />
)}

{granted && (
  <>
    <NavRow label={n.reminder} value={reminderLabel(reminderMinutes)} onPress={() => setOpen((o) =>
!o)} />
    {open && (
      <Choice
        options={REMINDER_CHOICES.map((m) => ({ value: String(m), label: reminderLabel(m) })))}
        value={String(reminderMinutes)}
        onChange={(v) => {
          setReminderMinutes(Number(v));
          setOpen(false);
        }}
      />
    )}
    {/* на iPhone вибрацията се определя от настройките на телефона */}
    {Platform.OS === 'android' && <SwitchRow label={n.vibrate} value={vibrate} onChange={setVibrate}
/>}
    <NavRow label={n.diagnostics} onPress={() => router.push('/diagnostics')} />
  </>
)}
</Group>
);
}

/** Ред със състояние: „Разрешени“ или (в оранжево) „Изключени“ + обяснение и връзка. */
function StatusRow({
  label,
  ok,
  okText,
  badText,
  note,
  action,
}: {
  label: string;
  ok: boolean;
  okText: string;
  badText: string;
  note: string;
  action: { label: string; onPress: () => void };
}) {
  if (ok) {
    return (

```

```

    <Row>
      <Text style={settingsStyles.label}>{label}</Text>
      <Text style={styles.value}>{okText}</Text>
    </Row>
  );
}
return (
  <>
    <Row>
      <View style={styles.exactText}>
        <View style={styles.exactHead}>
          <Text style={settingsStyles.label}>{label}</Text>
          <Text style={[styles.value, styles.warn]}>{badText}</Text>
        </View>
        <Text style={styles.small}>{note}</Text>
      </View>
    </Row>
    <Pressable onPress={action.onPress} accessibilityRole="button" style={styles.linkRow}>
      <Text style={styles.link}>{action.label}</Text>
    </Pressable>
  </>
);
}

const styles = StyleSheet.create({
  text: { flex: 1, fontFamily: fonts.regular, fontSize: 13, lineHeight: 18, color: colors.textDim },
  value: { fontFamily: fonts.semibold, fontSize: 13.5, color: colors.textDim },
  warn: { color: colors.warn },
  small: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted },
  exactText: { flex: 1, gap: 4 },
  exactHead: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  linkRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
});

```

BatteryWarning.tsx src/components/settings

- Жълтото поле най-горе в „Настройки“, докато телефонът ограничава Езан.

src/components/settings/BatteryWarning.tsx

60 реда

```

import { router } from 'expo-router';
import { Pressable, StyleSheet, Text, View } from 'react-native';

import { useI18n } from '@/i18n';
import { useNotificationStatus } from '@/services/notifications';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

/**
 * Предупреждението най-горе в „Настройки“ (тоскур v2): докато телефонът ограничава
 * Езан на заден план, алармите може да закъснеят. „Поправи“ → инструкциите за марката.
 */
export function BatteryWarning() {
  const { t } = useI18n();

```

```

const n = t.notifications;
const show = useNotificationStatus((s) => s.permission === 'granted' && s.battery === false);
if (!show) return null;
return (
  <View style={styles.box}>
    <Text style={styles.icon}>▲</Text>
    <View style={styles.text}>
      <Text style={styles.title}>{n.batteryTitle}</Text>
      <Text style={styles.body}>{n.batteryText}</Text>
      <Pressable
        onPress={() => router.push('/battery')}
        accessibilityRole="button"
        style={({ pressed }) => [styles.fix, pressed && styles.pressed]}
      >
        <Text style={styles.fixText}>{n.batteryFix}</Text>
      </Pressable>
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  box: {
    flexDirection: 'row',
    gap: 12,
    padding: 14,
    borderRadius: 18,
    backgroundColor: 'rgba(227,154,75,0.12)',
    borderWidth: 1,
    borderColor: 'rgba(227,154,75,0.45)',
  },
  icon: { fontSize: 18, color: colors.warn, marginTop: -1 },
  text: { flex: 1, gap: 2 },
  title: { fontFamily: fonts.bold, fontSize: 14, color: colors.warn },
  body: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.textDim },
  fix: {
    alignSelf: 'flex-start',
    marginTop: 8,
    paddingHorizontal: 14,
    paddingVertical: 7,
    borderRadius: 999,
    backgroundColor: colors.warn,
  },
  pressed: { opacity: 0.8 },
  fixText: { fontFamily: fonts.extrabold, fontSize: 13, color: '#1F1206' },
});

```

battery.tsx src/app

- Марката се познава сама; стъпките за Samsung, Xiaomi, Huawei/Honor, OPPO/Realme/OnePlus и други.
- „Отвори настройките на Езан“, а при връщане – проверка наново („Сега: без ограничения ✓“).

src/app/battery.tsx

210 реда

```

import { router } from 'expo-router';
import { useEffect, useRef, useState } from 'react';
import { AppState, Linking, Platform, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import {

```

```

    isIgnoringBatteryOptimizations,
    openAppSettings,
    openBatteryOptimizationSettings,
  } from '../modules/adhan-native';

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronIcon } from '@components/icons';
import { BATTERY_BRANDS, batteryBrand, type BatteryBrand } from '@domain/device';
import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const constants = Platform.constants as { Manufacturer?: string; Brand?: string; Model?: string };
const DETECTED: BatteryBrand = batteryBrand(constants.Manufacturer, constants.Brand);
/** „Samsung Galaxy A54“ – производителят с главна буква и моделът. */
const PHONE = [constants.Manufacturer, constants.Model]
  .filter(Boolean)
  .map((s) => s!.charAt(0).toUpperCase() + s!.slice(1))
  .join(' ');

/**
 * „Работа на заден план“: как да се махне оптимизацията на батерията за Езан –
 * стъпките според марката на телефона (Samsung, Xiaomi, Huawei, OPPO, друг).
 * Без това алармите на някои телефони закъсняват или не звънят.
 */
export default function BatteryScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const b = t.battery;
  const [brand, setBrand] = useState<BatteryBrand>(DETECTED);
  const [ok, setOk] = useState(() => isIgnoringBatteryOptimizations());

  // след връщане от настройките на телефона – проверява наново
  useEffect(() => {
    const sub = AppState.addEventListener('change', (state) => {
      if (state === 'active') setOk(isIgnoringBatteryOptimizations());
    });
    return () => sub.remove();
  }, []);

  const closing = useRef(false);
  const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
  };

  return (
    <View style={styles.root}>
      <GeometricPattern />
      <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
        <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
          <ChevronIcon direction="left" size={18} color={colors.gold} />
          <Text style={styles.backText}>{t.settings.title}</Text>
        </Pressable>
      </View>
      <ScrollView
        contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]
        showsVerticalScrollIndicator={false}
      >

```

```

<Text style={styles.title}>{b.title}</Text>
<Text style={styles.lede}>{b.lede}</Text>

{PHONE ? (
  <View style={styles.phone}>
    <Text style={styles.phoneLabel}>{b.yourPhone}</Text>
    <Text style={styles.phoneName}>{PHONE}</Text>
  </View>
) : null}

<View style={styles.steps}>
  {b.steps[brand].map(([text, path, note], i) => (
    <View key={`_${brand}-${i}`} style={styles.step}>
      <View style={styles.num}>
        <Text style={styles.numText}>{i + 1}</Text>
      </View>
      <Text style={styles.stepText}>
        {text}
        {path ? (
          <>
            {' '}
            <Text style={styles.path}> {path} </Text>
          </>
        ) : null}
        {note ? <Text style={styles.note}>{'\n'}{note}</Text> : null}
      </Text>
    </View>
  )]}
</View>

<Pressable
  onPress={openAppSettings}
  accessibilityRole="button"
  style={({ pressed }) => [styles.button, pressed && styles.pressed]}
>
  <Text style={styles.buttonText}>{b.open}</Text>
</Pressable>
{brand === 'other' && (
  <Pressable onPress={openBatteryOptimizationSettings} accessibilityRole="button" style=
{styles.secondary}>
    <Text style={styles.link}>{b.openList}</Text>
  </Pressable>
)}

<View style={[styles.status, ok ? styles.statusOk : ok === false ? styles.statusBad : null]}>
  <Text style={styles.statusLabel}>{b.now}</Text>
  <Text style={[styles.statusValue, ok ? styles.okText : ok === false ? styles.badText : null]}>
    {ok ? b.ok : ok === false ? b.bad : b.unknown}
  </Text>
</View>

<Text style={styles.other}>{b.otherPhone}</Text>
<View style={styles.chips}>
  {BATTERY_BRANDS.map((x) => (
    <Pressable
      key={x}
      onPress={() => setBrand(x)}
      accessibilityRole="button"
      accessibilityState={{ selected: x === brand }}
      style={[styles.chip, x === brand && styles.chipOn]}
    >

```

```

        <Text style={styles.chipText, x === brand && styles.chipTextOn]}>{b.brands[x]}</Text>
      </Pressable>
    )}}
  </View>

  <Pressable onPress={() => Linking.openURL('https://dontkillmyapp.com')} accessibilityRole="link">
    <Text style={styles.more}>{b.more}</Text>
  </Pressable>
</ScrollView>
</View>
);
}

const GREEN = '#5FBF8F';

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: { paddingHorizontal: 16, paddingBottom: 4 },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6, alignSelf: 'flex-start' },
  backText: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 12 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginTop: 4 },
  lede: { fontFamily: fonts.regular, fontSize: 13.5, lineHeight: 19.5, color: colors.textDim },
  phone: {
    padding: 14,
    borderRadius: 16,
    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.07)',
    gap: 2,
  },
  phoneLabel: { fontFamily: fonts.regular, fontSize: 12.5, color: colors.muted },
  phoneName: { fontFamily: fonts.bold, fontSize: 15, color: colors.text },
  steps: { gap: 4 },
  step: { flexDirection: 'row', gap: 12, paddingVertical: 8 },
  num: {
    width: 26,
    height: 26,
    borderRadius: 13,
    backgroundColor: 'rgba(212,168,87,0.18)',
    alignItems: 'center',
    justifyContent: 'center',
  },
  numText: { fontFamily: fonts.extrabold, fontSize: 13, color: colors.gold, includeFontPadding: false },
  stepText: { flex: 1, fontFamily: fonts.regular, fontSize: 14, lineHeight: 21, color: colors.text },
  path: { fontFamily: fonts.bold, backgroundColor: 'rgba(255,255,255,0.08)', color: colors.text },
  note: { fontFamily: fonts.regular, fontSize: 12.5, color: colors.muted },
  button: {
    height: 52,
    borderRadius: 999,
    backgroundColor: colors.gold,
    alignItems: 'center',
    justifyContent: 'center',
    marginTop: 4,
  },
  pressed: { opacity: 0.8 },
  buttonText: { fontFamily: fonts.extrabold, fontSize: 15, color: colors.goldInk },
  secondary: { alignItems: 'center', paddingVertical: 6 },
  link: { fontFamily: fonts.bold, fontSize: 14, color: colors.gold },
  status: {

```

```

flexDirection: 'row',
justifyContent: 'space-between',
paddingHorizontal: 14,
paddingVertical: 12,
borderRadius: 14,
backgroundColor: 'rgba(255,255,255,0.05)',
borderWidth: 1,
borderColor: 'rgba(255,255,255,0.08)',
},
statusOk: { backgroundColor: 'rgba(95,191,143,0.12)', borderColor: 'rgba(95,191,143,0.4)' },
statusBad: { backgroundColor: 'rgba(227,154,75,0.12)', borderColor: 'rgba(227,154,75,0.45)' },
statusLabel: { fontFamily: fonts.semibold, fontSize: 13.5, color: colors.text },
statusValue: { fontFamily: fonts.bold, fontSize: 13.5, color: colors.muted },
okText: { color: GREEN },
badText: { color: colors.warn },
other: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted, marginTop: 8,
paddingLeft: 4 },
chips: { flexDirection: 'row', flexWrap: 'wrap', gap: 6 },
chip: { paddingHorizontal: 12, paddingVertical: 7, borderRadius: 999, backgroundColor:
'rgba(255,255,255,0.07)' },
chipOn: { backgroundColor: colors.text },
chipText: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.textDim },
chipTextOn: { color: colors.base },
more: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.muted, textAlign: 'center', marginTop:
10 },
});

```

device.ts src/domain

- Марката по производителя (Build.MANUFACTURER / BRAND).

src/domain/device.ts

17 реда

```

/**
 * Марката на телефона – за инструкциите „Работа на заден план“.
 * Всяка марка крие настройките за батерията на различно място.
 */
export type BatteryBrand = 'samsung' | 'xiaomi' | 'huawei' | 'oppo' | 'other';

export const BATTERY_BRANDS: readonly BatteryBrand[] = ['samsung', 'xiaomi', 'huawei', 'oppo', 'other'];

/** По производителя и марката от Android (Build.MANUFACTURER / Build.BRAND). */
export function batteryBrand(manufacturer?: string | null, brand?: string | null): BatteryBrand {
  const m = `${manufacturer ?? ''} ${brand ?? ''}`.toLowerCase();
  if (m.includes('samsung')) return 'samsung';
  if (m.includes('xiaomi') || m.includes('redmi') || m.includes('poco')) return 'xiaomi';
  if (m.includes('huawei') || m.includes('honor')) return 'huawei';
  if (m.includes('oppo') || m.includes('realme') || m.includes('oneplus')) return 'oppo';
  return 'other';
}

```

Стъпка 6

Huawei без Google услуги и градовете по света

expo-location взима позицията през Google Play Services – на Huawei без тях не работи.

location.ts src/services

- Без Google услуги – позицията от LocationManager (GPS + мрежа) на Kotlin модула.
- И на другите Android, ако expo-location не успее – същият резервен път.
- Името извън България: от телефона, а без интернет или без геокодер – от офлайн списъка.

src/services/location.ts

122 реда

```
import * as Location from 'expo-location';
import { Platform } from 'react-native';
import { create } from 'zustand';

import { getNativeLocation, hasGoogleServices } from '../../modules/adhan-native';

import { nameFromAddress } from '@domain/abroadName';
import { isInBulgaria } from '@domain/places';
import { locationFromCoords, type PlaceName } from '@domain/resolve';
import { offlinePlaceName } from '@domain/worldCities';
import { useSettings } from '@store/settings';

/**
 * Определяне на мястото с GPS. Само докато приложението е отворено –
 * без фонов достъп до местоположението.
 */
export type LocationStatus = 'idle' | 'locating' | 'ok' | 'denied' | 'unavailable';

export const useLocationStatus = create<{ status: LocationStatus; lastFix: number }>(() => ({
  status: 'idle',
  lastFix: 0,
}));

const setStatus = (status: LocationStatus) => useLocationStatus.setState({ status });

function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T> {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => reject(new Error('timeout')), ms);
    promise.then(
      (v) => {
        clearTimeout(timer);
        resolve(v);
      },
      (e) => {
        clearTimeout(timer);
        reject(e);
      },
    );
  });
}

/**
 * Името на мястото извън България: от телефона (изисква интернет) – градът, а под него
 * районът и държавата (виж nameFromAddress). Без връзка или без Google услуги (Huawei) –
 * от офлайн списъка на градовете: „Istanbul“ / „Турция“.
 */
async function nameAbroad(
  latitude: number,
  longitude: number,
): Promise<{ name: PlaceName | null; country: string | null }> {
  try {
    const [addr] = await withTimeout(Location.reverseGeocodeAsync({ latitude, longitude }), 8000);
```

```

    const name = nameFromAddress(addr);
    if (name) return { name, country: addr?.isoCountryCode ?? null };
  } catch {
    // без интернет / без геокодер - офлайн
  }
  return { name: offlinePlaceName(latitude, longitude), country: null };
}

/**
 * Позицията: през Google Play Services (expo-location), а на телефони без тях (Huawei) -
 * от вградената в Android LocationManager. Ако expo-location не успее, и на други Android
 * телефони се пробва LocationManager. Накрая - последната известна позиция.
 */
async function getPosition(): Promise<{ latitude: number; longitude: number } | null> {
  if (Platform.OS === 'android' && hasGoogleServices() === false) {
    const native = await getNativeLocation(15000);
    return native ? { latitude: native.latitude, longitude: native.longitude } : null;
  }
  try {
    const p = await withTimeout(Location.getCurrentPositionAsync({ accuracy: Location.Accuracy.Balanced }),
15000);
    return p.coords;
  } catch {
    if (Platform.OS === 'android') {
      const native = await getNativeLocation(10000);
      if (native) return { latitude: native.latitude, longitude: native.longitude };
    }
    const last = await Location.getLastKnownPositionAsync().catch(() => null);
    return last?.coords ?? null;
  }
}

let running: Promise<void> | null = null;

/** Взима текущото местоположение и го записва като мястото от GPS. Паралелни извиквания се обединяват. */
export function refreshLocation(): Promise<void> {
  if (!running) {
    running = doRefresh().finally(() => {
      running = null;
    });
  }
  return running;
}

async function doRefresh(): Promise<void> {
  setStatus('locating');
  try {
    let perm = await Location.getForegroundPermissionsAsync();
    if (perm.status !== 'granted' && perm.canAskAgain) {
      perm = await Location.requestForegroundPermissionsAsync();
    }
    if (perm.status !== 'granted') {
      setStatus('denied');
      return;
    }
  }

  // точност ~100 м стига, за да се различат съседни села
  const position = await getPosition();
  if (!position) {
    setStatus('unavailable');
    return;
  }
}

```

```

    }

    const { latitude, longitude } = position;
    const abroad = isInBulgaria(latitude, longitude) ? null : await nameAbroad(latitude, longitude);
    useSettings.getState().setGpsLocation(LocationFromCoords(latitude, longitude, abroad?.name,
    abroad?.country));
    useLocationStatus.setState({ status: 'ok', lastFix: Date.now() });
  } catch {
    setStatus('unavailable');
  }
}

```

NativeLocation.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Последната позиция, ако е по-нова от 2 мин.; иначе първият отговор от мрежата или GPS (до 15 сек.).

modules/adhan-native/android/src/main/java/expo/modules/adhannative/NativeLocation.kt

123 реда

```

package expo.modules.adhannative

import android.Manifest
import android.content.Context
import android.content.pm.PackageManager
import android.location.Location
import android.location.LocationListener
import android.location.LocationManager
import android.os.Bundle
import android.os.Handler
import android.os.Looper

/**
 * Местоположение без Google услуги - с вградения в Android LocationManager (GPS + мрежа).
 * expo-location ползва Google Play Services и на Huawei без тях не работи;
 * затова там приложението вика това.
 */
object NativeLocation {
    /** Последна позиция, по-нова от това, се връща веднага. */
    private const val FRESH_MS = 2 * 60 * 1000L

    fun hasPermission(context: Context): Boolean =
        context.checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION) ==
        PackageManager.PERMISSION_GRANTED ||
        context.checkSelfPermission(Manifest.permission.ACCESS_COARSE_LOCATION) ==
        PackageManager.PERMISSION_GRANTED

    /** Има ли Google Play Services (инсталирани и включени). */
    fun hasGoogleServices(context: Context): Boolean =
        try {
            context.packageManager.getApplicationInfo("com.google.android.gms", 0).enabled
        } catch (e: PackageManager.NameNotFoundException) {
            false
        }

    fun toMap(l: Location): Map<String, Any?> = mapOf(
        "latitude" to l.latitude,
        "longitude" to l.longitude,
        "accuracy" to (if (l.hasAccuracy()) l.accuracy.toDouble() else null),
        "timestamp" to l.time.toDouble(),
        "provider" to l.provider,
    )
}

```

```

/** Най-новата известна позиция от всички източници (може да е стара). */
@Suppress("MissingPermission")
fun lastKnown(context: Context): Location? {
    if (!hasPermission(context)) return null
    val lm = context.getSystemService(Context.LOCATION_SERVICE) as LocationManager
    return lm.getProviders(true)
        .mapNotNull { p ->
            try {
                lm.getLastKnownLocation(p)
            } catch (e: Exception) {
                null
            }
        }
        .maxByOrNull { it.time }
}

/**
 * Текущата позиция: първият отговор от мрежата или GPS (по-бързият), иначе -
 * след `timeoutMs` последната известна. `done(null)` - няма позиция.
 */
@Suppress("MissingPermission", "DEPRECATION")
fun current(context: Context, timeoutMs: Long, done: (Location?) -> Unit) {
    if (!hasPermission(context)) {
        done(null)
        return
    }
    val last = lastKnown(context)
    if (last != null && System.currentTimeMillis() - last.time < FRESH_MS) {
        done(last)
        return
    }
    val lm = context.getSystemService(Context.LOCATION_SERVICE) as LocationManager
    val providers = listOf(LocationManager.NETWORK_PROVIDER, LocationManager.GPS_PROVIDER)
        .filter {
            try {
                lm.isProviderEnabled(it)
            } catch (e: Exception) {
                false
            }
        }
    if (providers.isEmpty()) {
        done(last)
        return
    }

    val main = Handler(Looper.getMainLooper())
    var finished = false
    val listeners = mutableListOf<LocationListener>()
    lateinit var timeout: Runnable

    fun finish(l: Location?) {
        if (finished) return
        finished = true
        main.removeCallbacks(timeout)
        listeners.forEach {
            try {
                lm.removeUpdates(it)
            } catch (e: Exception) {
                // нищо
            }
        }
    }

```

```

    }
    done(l)
}
timeout = Runnable { finish(lastKnown(context) ?: last) }

for (p in providers) {
    val listener = object : LocationListener {
        override fun onLocationChanged(location: Location) = finish(location)
        override fun onStatusChanged(provider: String?, status: Int, extras: Bundle?) {}
        override fun onProviderEnabled(provider: String) {}
        override fun onProviderDisabled(provider: String) {}
    }
    listeners.add(listener)
    try {
        lm.requestLocationUpdates(p, 0L, 0f, listener, Looper.getMainLooper())
    } catch (e: Exception) {
        // този източник не става - другият може
    }
}
main.postDelayed(timeout, timeoutMs)
}
}

```

worldCities.ts src/domain

- ~25 500 града (над 15 000 жители + всички столици) – ~600 KB, зарежда се при първа нужда.
- Името е на по-големия град, ако си в обхвата му: в центъра на Истанбул – „Istanbul“, не кварталът „Eminönü“.

src/domain/worldCities.ts

121 реда

```

import type { PlaceName } from './resolve';

/**
 * Офлайн списъкът на градовете по света (без България): ~25 000 града с над 15 000 жители
 * и всички столици/областни центрове – от GeoNames (CC BY 4.0), виж tools/build-world-cities.mjs.
 *
 * За какво е:
 * - името на мястото извън България, когато телефонът не може да го даде (без интернет,
 *   Huawei без Google услуги);
 * - държавата – по нея се избира методът за изчисление (виж methodForCountry).
 */

interface WorldData {
    names: string[];
    cc: string;
    lat: number[];
    lon: number[];
    countries: Record<string, [string, string]>;
}

let data: WorldData | null = null;

/** Зарежда се при първа нужда (~600 KB) – не забавя стартирането. */
function load(): WorldData {
    if (!data) {
        const raw = require('@data/world-cities.json') as {
            names: string;
            cc: string;
            lat: number[];

```

```

    lon: number[];
    countries: Record<string, [string, string]>;
  };
  data = { names: raw.names.split('\n'), cc: raw.cc, lat: raw.lat, lon: raw.lon, countries: raw.countries
};
}
return data;
}

export interface WorldCity {
  name: string;
  /** ISO код на държавата: „TR“ */
  country: string;
  latitude: number;
  longitude: number;
  /** Разстояние до точката, км. */
  km: number;
}

/**
 * Докъде „стига“ градът (км) според мястото му по население: в центъра на Истанбул
 * или Берлин името е „Istanbul“ / „Berlin“, а не кварталът (Eminönü, Mitte), който е по-близо.
 */
function reachKm(rank: number): number {
  if (rank < 100) return 25; // мегаполиси
  if (rank < 1000) return 12;
  if (rank < 5000) return 6;
  return 3;
}

/**
 * Градът на точката (до `maxKm`): най-големият град, в чийто обхват е точката,
 * иначе най-близкият. Списъкът е подреден по население.
 */
export function nearestCity(lat: number, lon: number, maxKm = 50): WorldCity | null {
  const d = load();
  const cosLat = Math.cos((lat * Math.PI) / 180);
  const kmTo = (i: number) => {
    const dy = d.lat[i] / 100 - lat;
    let dLon = d.lon[i] / 100 - lon;
    if (dLon > 180) dLon -= 360;
    else if (dLon < -180) dLon += 360;
    const dx = dLon * cosLat;
    return 111.195 * Math.sqrt(dx * dx + dy * dy);
  };
  let best = -1;
  let bestKm = Infinity;
  for (let i = 0; i < d.names.length; i++) {
    const km = kmTo(i);
    // първият (най-големият) град, който „покрива“ точката, печели веднага
    if (km <= reachKm(i)) {
      best = i;
      bestKm = km;
      break;
    }
    if (km < bestKm) {
      bestKm = km;
      best = i;
    }
  }
  if (best < 0 || bestKm > maxKm) return null;
}

```

```

    return {
      name: d.names[best],
      country: d.cc.slice(best * 2, best * 2 + 2),
      latitude: d.lat[best] / 100,
      longitude: d.lon[best] / 100,
      km: bestKm,
    };
  }

  /** Държавата на точката – по най-близкия град (до 600 км: и в открито море, и в пустинята). */
  export function countryAt(lat: number, lon: number): string | null {
    return nearestCity(lat, lon, 600)?.country ?? null;
  }

  /** „Турция“ / „Türkiye“; непозната – самият код. */
  export function countryName(code: string, lang: 'bg' | 'en'): string {
    const n = load().countries[code.toUpperCase()];
    return n ? n[lang === 'bg' ? 0 : 1] : code.toUpperCase();
  }

  /**
   * Име на мястото без интернет: градът и държавата („Istanbul“ / „Турция“).
   * null – няма град наблизо (тогава остават координатите).
   */
  export function offlinePlaceName(lat: number, lon: number): PlaceName | null {
    const city = nearestCity(lat, lon);
    if (!city) return null;
    return {
      names: { bg: city.name, en: city.name },
      detail: { bg: countryName(city.country, 'bg'), en: countryName(city.country, 'en') },
    };
  }
}

```

build-world-cities.mjs tools

- Как се прави списъкът от GeoNames (CC BY 4.0) – пуска се рядко. GeoNames е добавен в „За приложениято“.

tools/build-world-cities.mjs

58 реда

```

/**
 * Офлайн списъкът на градовете по света (извън България) – src/data/world-cities.json.
 *
 * Източник: GeoNames (CC BY 4.0) през npm пакета all-the-cities (градове с поне 1000 жители).
 * Вземат се градовете с поне 15 000 жители и всички столици/областни центрове.
 * Имената на държавите на български и английски – от Intl.DisplayNames на Node.
 *
 * Пускане (веднъж, когато трябва да се обнови):
 * npm i --no-save all-the-cities@3.1.0 pbf@3
 * node tools/build-world-cities.mjs
 */
import { createRequire } from 'node:module';
import { writeFileSync } from 'node:fs';

const require = createRequire(import.meta.url);
const all = require('all-the-cities');

const MIN_POPULATION = 15000;
const keep = all.filter(
  (c) =>

```

```

    c.country !== 'BG' && // България е с подробния списък на селищата (places.json)
    c.name &&
    (c.population >= MIN_POPULATION || c.featureCode === 'PPLC' || c.featureCode === 'PPLA'),
  );
// по-големите първо - при еднакво разстояние печели по-известният
keep.sort((a, b) => b.population - a.population);

const names = [];
let cc = '';
const lat = [];
const lon = [];
const countries = new Set();
for (const c of keep) {
  names.push(c.name.replace(/\n/g, ' '));
  cc += c.country;
  lon.push(Math.round(c.loc.coordinates[0] * 100));
  lat.push(Math.round(c.loc.coordinates[1] * 100));
  countries.add(c.country);
}

const bg = new Intl.DisplayNames(['bg'], { type: 'region' });
const en = new Intl.DisplayNames(['en'], { type: 'region' });
const countryNames = {};
for (const code of [...countries].sort()) {
  countryNames[code] = [bg.of(code) ?? code, en.of(code) ?? code];
}

const out = {
  source: 'GeoNames (CC BY 4.0) via all-the-cities',
  count: names.length,
  names: names.join('\n'),
  cc,
  lat,
  lon,
  countries: countryNames,
};

writeFileSync(new URL('../src/data/world-cities.json', import.meta.url), JSON.stringify(out));
console.log(`${names.length} cities, ${countries.size} countries`);

```

Стъпка 7

Метод според държавата

Извън България методът по подразбиране вече е „Автоматично“. Ръчният избор остава и има предимство.

ДЪРЖАВА	МЕТОД
Турция и Европа (без Франция, Великобритания, Ирландия)	Диянет
Франция	UOIF (12°/12°)
Великобритания, Ирландия	Moonsighting Committee
САЩ, Канада	ISNA
Саудитска Арабия	Umm al-Qura
Египет · Пакистан, Индия, Бангладеш, Афганистан	Египетски · Карачи
Русия · Малайзия · Индонезия	ДУМ (16°/15°) · JAKIM · Kemenag (20°/18°)
ОАЕ · Кувейт · Катар · Сингапур · Иран	собствените им
Другаде	Muslim World League

calc.ts src/domain

- 16 метода: тези от adhan + Франция, Русия, Малайзия, Индонезия със собствени ъгли.
- `methodForCountry()` – таблицата горе.

src/domain/calc.ts

193 реда

```
import {
  CalculationMethod,
  Coordinates,
  HighLatitudeRule,
  Madhab,
  PolarCircleResolution,
  PrayerTimes,
} from 'adhan';

import type { AppLocation } from '../location';
import { PRAYER_IDS, type PrayerTime } from '../prayers';

/** Методите, които се предлагат в настройките (извън България). */
export const CALC_METHODS = [
  'Turkey',
  'MuslimWorldLeague',
  'NorthAmerica',
  'UmmAlQura',
  'Egyptian',
  'Karachi',
  'MoonsightingCommittee',
  'Dubai',
  'Kuwait',
  'Qatar',
  'Singapore',
  'Tehran',
  'France',
  'Russia',
  'Malaysia',
  'Indonesia',
] as const;
export type CalculationMethodId = (typeof CALC_METHODS)[number];
```

```

/** В настройките: конкретен метод или „Автоматично“ – според държавата. */
export type MethodChoice = CalculationMethodId | 'auto';

/**
 * Методите, които ги няма в adhan – ъглите на Слънцето под хоризонта за Фаджр и Иша.
 * Франция – UQIF (12°/12°); Русия – Духовното управление на мюсюлманите (16°/15°);
 * Малайзия – JAKIM и Индонезия – Кетенаг (20°/18°).
 */
const CUSTOM_ANGLES: Partial<Record<CalculationMethodId, [number, number]>> = {
  France: [12, 12],
  Russia: [16, 15],
  Malaysia: [20, 18],
  Indonesia: [20, 18],
};

function methodParams(method: CalculationMethodId) {
  const custom = CUSTOM_ANGLES[method];
  if (custom) {
    const params = CalculationMethod.Other();
    params.fajrAngle = custom[0];
    params.ishaAngle = custom[1];
    return params;
  }
  return CalculationMethod[method as Exclude<CalculationMethodId, 'France' | 'Russia' | 'Malaysia' | 'Indonesia'>]();
}

/** Европа (без Франция, Великобритания и Ирландия): Диянет – както в джамиите на Диянет в Европа. */
const EUROPE_DIYANET = new Set(
  'AD AL AT AZ BA BE BY CH CY CZ DE DK EE ES FI GE GR HR HU IS IT LI LT LU LV MC MD ME MK MT NL NO PL PT RO RS SE SI SK SM UA VA XK TR'.split(
    ' ',
  ),
);

const BY_COUNTRY: Record<string, CalculationMethodId> = {
  US: 'NorthAmerica',
  CA: 'NorthAmerica',
  SA: 'UmmAlQura',
  EG: 'Egyptian',
  PK: 'Karachi',
  IN: 'Karachi',
  BD: 'Karachi',
  AF: 'Karachi',
  FR: 'France',
  GB: 'MoonsightingCommittee',
  IE: 'MoonsightingCommittee',
  RU: 'Russia',
  MY: 'Malaysia',
  BN: 'Malaysia',
  ID: 'Indonesia',
  AE: 'Dubai',
  KW: 'Kuwait',
  QA: 'Qatar',
  SG: 'Singapore',
  IR: 'Tehran',
};

/** Методът по подразбиране за държавата (ISO код). Непозната – Muslim World League. */
export function methodForCountry(country: string | null | undefined): CalculationMethodId {
  const c = country?.toUpperCase() ?? '';

```

```

    if (BY_COUNTRY[c]) return BY_COUNTRY[c];
    if (EUROPE_DIYANET.has(c)) return 'Turkey';
    return 'MuslimWorldLeague';
}

export type AsrMadhab = 'shafi' | 'hanafi';

/** Правило за северни ширини – как се смятат Фаджр и Иша, когато няма истински здрач. */
export const HIGH_LAT_RULES = ['seventhofthenight', 'middleofthenight', 'twilightangle'] as const;
export type HighLatRuleId = (typeof HIGH_LAT_RULES)[number];

/** Над тази ширина (на север или юг) настройката „северни ширини“ има значение и се показва. */
export const HIGH_LATITUDE = 48;

export function isHighLatitude(lat: number): boolean {
    return Math.abs(lat) > HIGH_LATITUDE;
}

/**
 * Над тази ширина има полярен ден и полярна нощ (Тромсьо, Норилск, Свалбард, полюсите).
 * В дните, в които изчислението за истинската ширина не става (няма изгрев/залез,
 * разбъркан ред), часовете са за същата дължина, но на 65° – най-близките места
 * с нормален ден и нощ („най-близката страна“, aqrab al-bilad).
 */
export const POLAR_LATITUDE = 65;

export function isPolar(lat: number): boolean {
    return Math.abs(lat) > POLAR_LATITUDE;
}

/**
 * Полярен ден или полярна нощ – слънцето този ден не изгрява или не залязва.
 * Деклинацията на слънцето е приблизителна ( $\pm 0,5^\circ$ ), затова има запас от 1,5°:
 * в граничните дни също се минава на 65°, вместо да се търсят часове от други дни
 * (това търсене в adhan е бавно – месец на Свалбард отнемаше ~0,5 сек. на компютър).
 */
export function isPolarDayOrNight(date: Date, lat: number): boolean {
    const start = new Date(date.getFullYear(), 0, 1);
    const dayOfYear = Math.round((date.getTime() - start.getTime()) / 86_400_000) + 1;
    const decl = 23.44 * Math.sin(((2 * Math.PI) / 365) * (284 + dayOfYear));
    return Math.abs(lat) > 90 - Math.abs(decl) - 1.5;
}

/** Всички часове са истински дати и вървят по ред: Фаджр < Изгрев < Зухр < Аср < Магриб < Иша. */
export function isValidDay(times: PrayerTime[]): boolean {
    return times.every(
        (p, i) => !isNaN(p.time.getTime()) && (i === 0 || p.time.getTime() > times[i - 1].time.getTime()),
    );
}

/**
 * Астрономическо изчисление с adhan – за места извън България.
 * Методът „Turkey“ (Диянет) вече съдържа предпазните минути (темкин) на Диянет.
 * Сравнено с календара на Мюфтийството за София, средната разлика е ~0,
 * но в отделни дни стига 2–4 мин. (виж docs/PLAN.md, етап 2).
 */
export function computeCalcDay(
    date: Date,
    location: AppLocation,
    method: CalculationMethodId,
    madhab: AsrMadhab,

```

```

    highLatRule: HighLatRuleId = 'seventhofthenight',
  ): PrayerTime[] {
    const lat = location.latitude;
    const sign = lat < 0 ? -1 : 1;
    // Полярен ден/нощ → направо 65°. Иначе първо истинската ширина; ако денят излезе
    // невалиден (липсващ час или разбъркан ред) – 65°, а ако и там не стане – още по-близо до екватора.
    let tryLat = isPolar(lat) && isPolarDayOrNight(date, lat) ? sign * POLAR_LATITUDE : lat;
    for (;;) {
      const times = calcAt(date, tryLat, location.longitude, method, madhab, highLatRule);
      if (isValidDay(times) || Math.abs(tryLat) <= HIGH_LATITUDE) return times;
      tryLat = sign * Math.min(Math.abs(tryLat) - 1, POLAR_LATITUDE);
    }
  }

function calcAt(
  date: Date,
  latitude: number,
  longitude: number,
  method: CalculationMethodId,
  madhab: AsrMadhab,
  highLatRule: HighLatRuleId,
): PrayerTime[] {
  const params = methodParams(method);
  params.madhab = madhab === 'hanafi' ? Madhab.Hanafi : Madhab.Shafi;
  if (isHighLatitude(latitude)) {
    params.highLatitudeRule =
      highLatRule === 'middleofthenight'
        ? HighLatitudeRule.MiddleOfTheNight
        : highLatRule === 'twilightangle'
        ? HighLatitudeRule.TwilightAngle
        : HighLatitudeRule.SeventhOfTheNight;
    // резерва за граничните дни около полярния кръг – най-близкият ден с нормални часове
    params.polarCircleResolution = PolarCircleResolution.AqrabYaum;
  }
  const coords = new Coordinates(latitude, longitude);
  // обяд, за да не може смяната на лятно/зимно време да измести датата
  const noon = new Date(date.getFullYear(), date.getMonth(), date.getDate(), 12);
  const pt = new PrayerTimes(coords, noon, params);
  return PRAYER_IDS.map((id) => ({ id, time: pt[id] }));
}

```

resolve.ts src/domain

- Мястото от GPS помни държавата; за старите записи тя се намира по координатите.

src/domain/resolve.ts

43 реда

```

import { formatCoords, type AppLocation } from './location';
import { isInBulgaria, placeAt, placeToLocation } from './places';
import { countryAt } from './worldCities';

export interface PlaceName {
  names: { bg: string; en: string };
  detail?: { bg: string; en: string };
}

/**
 * От координати (GPS) към място с правилния източник на часове:
 * - в България – населеното място, в което си (виж placeAt), и календарът на Мюфтийството;
 * - извън България – изчисление за точните координати; името идва отвън

```

```

* (обратно геокодиране от телефона), а без интернет – координатите.
*/
export function locationFromCoords(
  lat: number,
  lon: number,
  abroadName?: PlaceName | null,
  country?: string | null,
): AppLocation {
  if (isInBulgaria(lat, lon)) return placeToLocation(placeAt(lat, lon));
  const coords = formatCoords(lat, lon);
  return {
    id: `gps:${lat.toFixed(3)},${lon.toFixed(3)}`,
    names: abroadName?.names ?? { bg: coords, en: coords },
    detail: abroadName?.detail,
    latitude: lat,
    longitude: lon,
    source: 'calc',
    country: (country ?? countryAt(lat, lon) ?? undefined)?.toUpperCase(),
  };
}

const countryCache = new Map<string, string | null>();

/** Държавата на мястото: от записа, а в старите записи – по координатите (офлайн списъка). */
export function countryOf(location: AppLocation): string | null {
  if (location.source === 'mufti') return 'BG';
  if (location.country) return location.country;
  if (!countryCache.has(location.id)) countryCache.set(location.id, countryAt(location.latitude, location.longitude));
  return countryCache.get(location.id) ?? null;
}

```

settings.ts src/store

- `method: 'auto'` по подразбиране; версия 2 на записа: старият „Диянет“ по подразбиране става „Автоматично“.
- `resolveMethod()` – „Автоматично“ → методът за държавата.

src/store/settings.ts

176 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import {
  DEFAULT_HIGH_LAT_RULE,
  DEFAULT_LOCATION,
  DEFAULT_MADHAB,
  DEFAULT_METHOD,
} from '@config/defaults';
import {
  methodForCountry,
  type AsrMadhab,
  type CalculationMethodId,
  type HighLatRuleId,
  type MethodChoice,
} from '@domain/calc';
import type { AppLocation } from '@domain/location';
import { countryOf } from '@domain/resolve';
import type { PrayerId } from '@domain/prayers';

```

```

import type { TimesOptions } from '@domain/times';

/** Граници на ръчните корекции. */
export const OFFSET_LIMIT = 15; // ±15 мин. за молитва
export const HIJRI_LIMIT = 2; // ±2 дни
/** Възможните напомняния преди молитвата (минути); 0 = без напомняне. */
export const REMINDER_CHOICES = [0, 5, 10, 15, 20, 30] as const;

const NO_OFFSETS: Record<PrayerId, number> = {
  fajr: 0,
  sunrise: 0,
  dhuhr: 0,
  asr: 0,
  maghrib: 0,
  isha: 0,
};

const clamp = (v: number, limit: number) => Math.max(-limit, Math.min(limit, v));

interface SettingsState {
  /** true - мястото идва от GPS; false - избрано ръчно. */
  autoLocation: boolean;
  /** Последното място от GPS (пази се, за да работи веднага при следващо отваряне и без GPS). */
  gpsLocation: AppLocation | null;
  /** Ръчно избраното място. */
  manualLocation: AppLocation | null;

  /** 'auto' - според държавата (виж methodForCountry). */
  method: MethodChoice;
  madhab: AsrMadhab;
  highLatRule: HighLatRuleId;
  offsets: Record<PrayerId, number>;
  hijriAdjust: number;
  /** Напомняне X минути преди молитвата; 0 = без. */
  reminderMinutes: number;
  /** Вибрация при известие (Android; на iPhone решава телефонът). */
  vibrate: boolean;

  setAutoLocation: (on: boolean) => void;
  setGpsLocation: (loc: AppLocation) => void;
  chooseLocation: (loc: AppLocation) => void;
  setMethod: (m: MethodChoice) => void;
  setMadhab: (m: AsrMadhab) => void;
  setHighLatRule: (r: HighLatRuleId) => void;
  changeOffset: (id: PrayerId, delta: number) => void;
  resetOffsets: () => void;
  changeHijriAdjust: (delta: number) => void;
  setReminderMinutes: (n: number) => void;
  setVibrate: (on: boolean) => void;
}

export const useSettings = create<SettingsState>()(
  persist(
    (set) => ({
      autoLocation: true,
      gpsLocation: null,
      manualLocation: null,
      method: DEFAULT_METHOD,
      madhab: DEFAULT_MADHAB,
      highLatRule: DEFAULT_HIGH_LAT_RULE,
      offsets: NO_OFFSETS,
    })
  )
);

```

```

hijriAdjust: 0,
reminderMinutes: 0,
vibrate: true,

setAutoLocation: (on) => set({ autoLocation: on }),
setGpsLocation: (loc) => set({ gpsLocation: loc }),
// ръчният избор изключва GPS, за да не го „презapiше“ следващото определяне
chooseLocation: (loc) => set({ manualLocation: loc, autoLocation: false }),
setMethod: (method) => set({ method }),
setMadhab: (madhab) => set({ madhab }),
setHighLatRule: (highLatRule) => set({ highLatRule }),
changeOffset: (id, delta) =>
  set((s) => ({ offsets: { ...s.offsets, [id]: clamp(s.offsets[id] + delta, OFFSET_LIMIT) } })),
resetOffsets: () => set({ offsets: NO_OFFSETS }),
changeHijriAdjust: (delta) =>
  set((s) => ({ hijriAdjust: clamp(s.hijriAdjust + delta, HIJRI_LIMIT) })),
setReminderMinutes: (reminderMinutes) => set({ reminderMinutes }),
setVibrate: (vibrate) => set({ vibrate }),
}),
{
  name: 'ezan.settings',
  // 2 (етап 5): методът по подразбиране е „Автоматично“ – старият „Диянет“ по подразбиране
  // става „Автоматично“ (за Турция и Европа пак е Диянет)
  version: 2,
  migrate: (persisted, version) => {
    const state = persisted as Partial<SettingsState>;
    if (version < 2 && state.method === 'Turkey') state.method = 'auto';
    return state as SettingsState;
  },
  storage: createJSONStorage(() => AsyncStorage),
  // новите полета (reminderMinutes, vibrate) липсват в старите записи – тогава важат стойностите по
  // подразбиране
  partialize: ({
    autoLocation,
    gpsLocation,
    manualLocation,
    method,
    madhab,
    highLatRule,
    offsets,
    hijriAdjust,
    reminderMinutes,
    vibrate,
  }) => ({
    reminderMinutes,
    vibrate,
    autoLocation,
    gpsLocation,
    manualLocation,
    method,
    madhab,
    highLatRule,
    offsets,
    hijriAdjust,
  }),
},
),
);

/** Мястото, за което се показват часовете в момента. */
export function selectLocation(s: SettingsState): AppLocation {

```

```

const chosen = s.autoLocation ? s.gpsLocation : s.manualLocation;
return chosen ?? s.gpsLocation ?? DEFAULT_LOCATION;
}

type TimesParts = Pick<SettingsState, 'method' | 'madhab' | 'highLatRule' | 'offsets'> & { location:
AppLocation };

/** Опциите за часовете и техният „отпечатък“ `key` – по него се решава кога да се преизчисли. */
export function buildTimesOptions({ location, method, madhab, highLatRule, offsets }: TimesParts): {
  options: TimesOptions;
  key: string;
} {
  const options: TimesOptions = { location, method: resolveMethod(method, location), madhab, highLatRule,
offsets };
  const key = JSON.stringify([
    location.id,
    location.latitude,
    location.longitude,
    location.source,
    location.muftiShift,
    options.method,
    madhab,
    highLatRule,
    offsets,
  ]);
  return { options, key };
}

/** „Автоматично“ → методът за държавата на мястото; иначе избраният. */
export function resolveMethod(method: MethodChoice, location: AppLocation): CalculationMethodId {
  return method === 'auto' ? methodForCountry(countryOf(location)) : method;
}

/** Същото, направо от състоянието – без React (ползва се и от известията във фонов режим). */
export function selectTimesOptions(s: SettingsState): { options: TimesOptions; key: string } {
  return buildTimesOptions({ ...s, location: selectLocation(s) });
}

```

Стъпка 8

Тестове

201 теста общо: 43 нови в `stage5.test.ts`, а всички нови методи минават и през световните тестове (без грешни часове по целия свят).

stage5.test.ts src/domain/__tests__

- Градовете: Истанбул, Берлин, Париж, Одрин, Скопие, Мека; океанът; България не е в списъка.
- Методът за 20 държави; държавата на старите записи.
- UOIF, JAKIM и ДУМ спрямо Muslim World League; марките телефони.

`src/domain/__tests__/stage5.test.ts`

119 реда

```

import { computeCalcDay, methodForCountry } from '../calc';
import { batteryBrand } from '../device';
import { formatHM } from '../format';
import { countryOf, locationFromCoords } from '../resolve';
import { countryAt, countryName, nearestCity, offlinePlaceName } from '../worldCities';

```

```

describe('офлайн градовете по света', () => {
  it.each([
    ['центъра на Истанбул (не квартала Eminönü)', 41.0054, 28.9768, 'Istanbul', 'TR'],
    ['Кадъкьой – пак Истанбул', 40.99, 29.03, 'Istanbul', 'TR'],
    ['Берлин Мите (не квартала Mitte)', 52.52, 13.405, 'Berlin', 'DE'],
    ['Париж', 48.8566, 2.3522, 'Paris', 'FR'],
    ['Одрин', 41.6771, 26.5557, 'Edirne', 'TR'],
    ['Скопие', 41.9981, 21.4254, 'Skopje', 'MK'],
    ['Мека', 21.4225, 39.8262, 'Mecca', 'SA'],
  ])('%s', (_, lat, lon, name, cc) => {
    const c = nearestCity(lat, lon)!;
    expect(c.name).toBe(name);
    expect(c.country).toBe(cc);
  });

  it('в открития океан – няма град, няма държава', () => {
    expect(nearestCity(30, -40)).toBeNull();
    expect(countryAt(30, -40)).toBeNull();
  });

  it('България не е в списъка (там е подробният списък на селищата)', () => {
    expect(nearestCity(42.6977, 23.3217, 5)).toBeNull();
  });

  it('името без интернет: градът и държавата на езика на приложението', () => {
    expect(offLinePlaceName(41.0054, 28.9768)).toEqual({
      names: { bg: 'Istanbul', en: 'Istanbul' },
      detail: { bg: 'Турция', en: 'Türkiye' },
    });
    expect(countryName('DE', 'bg')).toBe('Германия');
    expect(countryName('XX', 'bg')).toBe('XX');
  });
});

describe('метод „Автоматично“ според държавата', () => {
  it.each([
    ['TR', 'Turkey'],
    ['DE', 'Turkey'],
    ['MK', 'Turkey'],
    ['FR', 'France'],
    ['GB', 'MoonsightingCommittee'],
    ['US', 'NorthAmerica'],
    ['CA', 'NorthAmerica'],
    ['SA', 'UmmAlQura'],
    ['EG', 'Egyptian'],
    ['PK', 'Karachi'],
    ['RU', 'Russia'],
    ['MY', 'Malaysia'],
    ['ID', 'Indonesia'],
    ['AE', 'Dubai'],
    ['KW', 'Kuwait'],
    ['QA', 'Qatar'],
    ['SG', 'Singapore'],
    ['IR', 'Tehran'],
    ['JP', 'MuslimWorldLeague'],
    [null, 'MuslimWorldLeague'],
  ])('%s → %s', (cc, method) => {
    expect(methodForCountry(cc)).toBe(method);
  });
});

```

```

it('мястото от GPS помни държавата; старите записи я намират по координатите', () => {
  const paris = locationFromCoords(48.8566, 2.3522, null);
  expect(paris.country).toBe('FR');
  expect(countryOf({ ...paris, country: undefined, id: 'old-paris' })).toBe('FR');
  expect(countryOf(locationFromCoords(42.6977, 23.3217))).toBe('BG');
});

describe('новите методи', () => {
  const day = new Date(2026, 9, 1, 12);
  const at = (lat: number, lon: number, method: Parameters<typeof computeCalcDay>[2]) =>
    Object.fromEntries(
      computeCalcDay(day, { id: 'x', names: { bg: '', en: '' }, latitude: lat, longitude: lon, source:
'calc' }, method, 'shafi').map(
        (p) => [p.id, p.time],
      ),
    );

  it('Франция (UOIF 12°): Фаджр в Париж е по-късно от Muslim World League (18°)', () => {
    const uoif = at(48.8566, 2.3522, 'France');
    const mwl = at(48.8566, 2.3522, 'MuslimWorldLeague');
    expect(uoif.fajr.getTime() - mwl.fajr.getTime()).toBeGreaterThan(20 * 60_000);
    expect(uoif.isha.getTime()).toBeLessThan(mwl.isha.getTime());
  });

  it('Малайзия и Индонезия (20°/18°): Фаджр по-рано от MWL (18°)', () => {
    const jakim = at(3.139, 101.6869, 'Malaysia');
    const mwl = at(3.139, 101.6869, 'MuslimWorldLeague');
    expect(jakim.fajr.getTime()).toBeLessThan(mwl.fajr.getTime());
  });

  it('Русия (16°/15°) – подредени часове в Москва', () => {
    const t = at(55.7558, 37.6173, 'Russia');
    const order = ['fajr', 'sunrise', 'dhuhr', 'asr', 'maghrib', 'isha'].map((k) => t[k].getTime());
    expect([...order].sort((a, b) => a - b)).toEqual(order);
    expect(formatHM(t.dhuhr)).toMatch(/^d\d:d\d$/);
  });

  describe('марката на телефона', () => {
    it.each([
      ['samsung', 'samsung', 'samsung'],
      ['Xiaomi', 'Redmi', 'xiaomi'],
      ['Xiaomi', 'POCO', 'xiaomi'],
      ['HUAWEI', 'HUAWEI', 'huawei'],
      ['HONOR', 'HONOR', 'huawei'],
      ['OnePlus', 'OnePlus', 'oppo'],
      ['realme', 'realme', 'oppo'],
      ['Google', 'google', 'other'],
      [undefined, undefined, 'other'],
    ])('%s / %s → %s', (m, b, expected) => {
      expect(batteryBrand(m, b)).toBe(expected);
    });
  });
});

```

Трябва да видиш: Tests: 201 passed, 201 total

npm test

Проверка и край на етапа

Етап 5 е готов, когато на телефона:

- **Пробна аларма:** Настройки → Проверка на известията → „Аларма“, заключи телефона. След 10 сек. се отваря екранът „Аларма“, езанът звучи докрай, вибрира силно.
- „Заглуши звука“ и бутонът за звука – тихо, бутонът става „Затвори“. „Спри езана“ – всичко изчезва.
- Докато ползваш телефона – известие отгоре със „Спри езана“ и „Заглуши“.
- Камбанка „аларма“ на молитва – в „Проверка на известията“ → „Следващи“ се вижда като „Аларма“; в часа ѝ звъни (и в „Последни“ – „навреме“).
- Рестартирай телефона – алармите остават в „Следващи“ и звънят.
- Android 14+: ако „Аларма на цял екран“ е изключена – разреши я от Настройки.
- „Работа на заден план“ – след стъпките за Samsung стойността е „Без ограничения“ и жълтото поле изчезва.
- Huawei без Google услуги (ако има как – или Huawei Cloud Debugging): мястото, известията и алармата.

Терминал

```
git add .  
git commit -m "Етап 5: пълен езан на Android"  
git tag stage-5
```

След това – етап 6 (звуци и мелодии, преслушване, лицензирани записи на езан).